

Block

2

Artificial Intelligence - Knowledge Representation

UNIT 4

Predicate and Propositional Logic	147
--	------------

UNIT 5

First Order Predicate Logic	181
------------------------------------	------------

UNIT 6

Rule Based Systems and Other Formalism	213
---	------------

BLOCK-2 INTRODUCTION

The Block-2, titled “**Artificial Intelligence - Knowledge Representation**,” is comprised of three units. The details are as follows:

- **Unit-4 Predicate and Propositional Logic**
- **Unit-5 First Order Logic**
- **Unit-6 Rule-based Systems and other formalisms**

Unit 4: introduces the basics of symbolic logic, covering propositional and predicate logic as key tools for formal reasoning and problem solving. It explains propositions, logical connectives, and inference rules, along with concepts like validity and satisfiability. The unit also introduces predicate logic with quantifiers and highlights proof techniques such as natural deduction and resolution, emphasizing the importance of logic in knowledge representation and artificial intelligence.

Unit 5: introduces First Order Predicate Logic (FOPL) as an extension of propositional logic to handle more expressive and meaningful reasoning. The unit focuses on the syntax and semantics of FOPL, interpretation of statements, and the role of quantifiers in expressing generalizations. It also covers inference and entailment, along with techniques such as conversion to clausal form, resolution, and unification, which are essential for automated reasoning and applications in artificial intelligence.

Unit 6: introduces the concept of rule-based systems and other knowledge representation formalisms used in artificial intelligence, with a focus on how knowledge can be structured and utilized for intelligent problem solving. It explains the fundamentals of expert systems and knowledge-based systems, highlighting the role of rules, facts, and inference mechanisms such as forward and backward chaining. The unit further explores alternative representation techniques including semantic networks, frames, and scripts, emphasizing their structure, features, and applications in representing real-world knowledge effectively for reasoning and decision-making

UNIT 4 PREDICATE AND PROPOSITIONAL LOGIC

Structure	Page No.
4.1 Introduction	147
4.2 Objectives	148
4.3 Introduction to Propositional Logic	148
4.4 Syntax of Propositional Logic	150
4.4.1 Atomic Propositions	150
4.4.2 Compound Propositions	150
4.5 Logical Connectives	152
4.5.1 Conjunction	152
4.5.2 Disjunction	152
4.5.3 Negation	152
4.5.4 Implication	153
4.5.5 Bi-Conditional	153
4.6 Semantics	154
4.6.1 Negation Truth Table	156
4.6.2 Conjunction/Disjunction/Implication/ Biconditional Truth Table	156
4.6.3 Truth Table with three variables	157
4.7 Propositional Rules of Inference	159
4.7.1 Modus Ponens (MP)	159
4.7.2 Modus Tollens (MT)	159
4.7.3 Disjunctive Syllogism (DS)	159
4.7.4 Addition	159
4.7.5 Simplification	160
4.7.6 Conjunction	160
4.7.7 Hypothetical Syllogism (HS)	160
4.7.8 Absorption	160
4.7.9 Constructive Dilemma (CD)	160
4.8 Propositional Rules of Replacement	160
4.9 Validity and Satisfiability	161
4.10 Introduction to Predicate Logic	162
4.11 Inferencing in Predicate Logic	164
4.12 Proof Systems	166
4.13 Natural Deduction	167
4.14 Propositional Resolution	171
4.14.1 Clausal Form	171
4.14.2 Determining Unsatisfiability	173
4.15 Answers/Solutions	178
4.16 Further Readings	180

4.1 INTRODUCTION

Logic is the study and analysis of the nature of a valid argument, the reasoning tool by which valid inferences can be drawn from a given set of facts and premises. It is the basis on which all the sciences are built, and this mathematical theory of logic is called symbolic logic. The English

mathematician George Boole (1815-1864) seriously studied and developed this theory, called symbolic logic.

The reason why the subject matter of the study is called Symbolic Logic is that symbols are used to denote facts about objects of the domain and relationships between these objects. Then the symbolic representations, rather than the original facts and relationships, are manipulated to draw conclusions or solve problems.

The basic building blocks of arguments in symbolic logic are declarative sentences, called propositions or statements. Logical rules have the power to give accuracy to mathematical statements. These rules come to the rescue when there is a need to differentiate between valid and invalid mathematical arguments. Symbolic logic may be thought of as a formal language for representing facts about objects and relationships between objects of a problem domain, along with a precise inferencing mechanism for reasoning and deduction.

Using symbolic logic, we can formalize our arguments and logical reasoning in a manner that can easily show if the reasoning is valid or is a fallacy. How we symbolize the reasoning is what is presented in this unit.

4.2 OBJECTIVES

After going through this unit, you should be able to:

1. Understand the meaning of propositional logic.
2. Differentiate between atomic and compound propositions.
3. Know different types of connectives, their associated semantics and corresponding truth tables.
4. Define propositional rules of inference and replacement.
5. Differentiate between valid and satisfiable arguments.

4.3 INTRODUCTION TO PROPOSITIONAL LOGIC

Apart from the application of logic in mathematics, it also helps in various other tasks related to computer science. It is widely used in the design of electronic circuitry, the programming of Android applications, and the application of artificial intelligence to various tasks. In simple terms, a proposition is a statement which is either true or false.

Consider the following statements:

1. Earth revolves around the sun.
2. Water freezes at 100° Celsius.
3. An hour has 3600 seconds.
4. 2 is the only even prime number.
5. Mercury is the closest planet to the Sun in the solar system.

6. The USA lies on the continent of North America.
7. $1 + 2 = 4$.
8. 15 is a prime number.
9. Moon rises in the morning and sets in the evening.
10. Delhi is the capital of India.

For all the above statements, one can easily conclude whether the particular statement is true or false, so these are propositions. The first statement is a universal truth. The second statement is false as the water freezes at 0° Celsius. The third statement is again a universal truth. The fourth statement is true. The fifth statement is also true, as it is again a universal truth. On similar lines, the sixth statement is also true. The seventh and eighth statements are false again, as they deny the basic mathematical rules. The ninth statement is a negation of the universal truth, so it is a false statement. The tenth Statement is also true.

Now consider the following statements:

1. What is your name?
2. $a + 5 = b$.
3. Who is the prime minister of India?
4. p is less than 5.
5. Pay full attention while you are in class.
6. Let's play football in the evening.
7. Don't behave like a child, you are grown up now!
8. How much do you earn?
9. $\angle X$ is an acute angle greater than 27° .

For all the above statements, we can't say anything about their truthfulness, so they are not propositions. First and third statements are neither true nor false as they are interrogative in nature. Also, we can't say anything about the truthfulness of the second statement until and unless we have the values of a and b . Similar reasoning applies to the fourth statement as well. We can't say anything about the fifth, sixth, and seventh statements again, as they are informative statements. Eighth statement is again interrogative in nature. Again, we can't say anything about the truthfulness of the ninth statement until and unless we have the value of $\angle X$.

Propositional logic has the following facts:

1. Propositional statements can be either true or false; they can't be both simultaneously.
2. Propositional logic is also referred to as binary logic as it works only on the two values 1 (True) and 0 (False).
3. Symbols or symbolic variables such as x, y, z, P, Q, R , etc. are used for representing the logic and propositions.
4. Any proposition or statement which is always valid (true) is known as a tautology.

5. Any proposition or statement which is always invalid (false) is known as a contradiction.
6. A table listing all the possible truth values of a proposition is known as a truth table.
7. Objects, relations (or functions) and logical connectives are the basic building blocks of propositional logic.
8. Logical connectives are also referred to as logical operators.
9. Statements which are interrogative, informative, or opinions such as “Where is Chandni Chowk located?”, “Mumbai is a good city to live in”, “Result will be declared on 31st March” are not propositions.

4.4 SYNTAX OF PROPOSITIONAL LOGIC

The syntax of propositional logic allows two types of sentences to represent knowledge. The two types are as follows:

4.4.1 Atomic Propositions

These are the simplest propositions containing a single proposition symbol and are either true or false. Some of the examples of atomic propositions are as follows:

1. “Venus is the closest planet to the Sun in the solar system” is an atomic proposition since it is a false fact.
2. “ $7 - 3 = 4$ ” is an atomic proposition as it is a true fact.

4.4.2 Compound Propositions

They are formed by a collection of atomic propositions joined with logical connectives or logical operators. Some of the examples of compound propositions are as follows:

1. The Sun is very bright today, and it's very hot outside.
2. Diana studies in class 8th and her school is in Karol Bagh.

☞ Check Your Progress 1

Which of the following statements are propositions? Write yes or no.

1. How are you?

.....
.....

2. Sachin Tendulkar is one of the best cricketers in India.

.....
.....

3. The honorable Ram Nath Kovind is the 10th and current president of India.

.....
.....
4. Lord Ram of the kingdom of Ayodhya is an example of a people's king.

.....
.....
5. In which year did Prophet Muhammad receive verbal revelations from Allah in the cave, Mount Hira, presently located in Saudi Arabia?

.....
.....
6. Akbar was the founder of the Mughal dynasty in India.

.....
.....
7. In the year 2019, renowned actor Shri Amitabh Bachchan was awarded with the Padma Vibhushan, which is the second-highest civilian honour of the Republic of India.

.....
.....
8. One should avoid eating fast food in order to maintain good health.

.....
.....
9. What is your age?

.....
.....
10. The first case of COVID-19 was detected in China.

.....
.....
11. Name the author of the book series "Shiva Trilogy".

.....
.....
12. Former Prime Minister of India, Shri Atal Bihari Vajpayee, was a member of which political party?

.....
.....
13. Wing Commander Rakesh Sharma is the only Indian citizen to travel in space to date.

.....
.....
14. Where do you live?

4.5 LOGICAL CONNECTIVES

Logical connectives are the operators used to join two or more atomic propositions (operands). The joining should be done in a way that the logic and truth value of the obtained compound proposition is dependent on the input atomic propositions and the connective used.

4.5.1 Conjunction

A proposition “ $A \wedge B$ ” with a connective $\wedge$ is known as *conjunction* of A and B. It is a proposition (or operation) which is true only when both the constituent propositions are true. Even if one of the input propositions is false, then the output is also false. It is also referred to as AND-ing the propositions. Example:

Ram is a playful boy and he loves to play football. It can be written as:

A = Ram is a playful boy.

B = Ram loves to play football.

$A \wedge B$ = Ram is a playful boy and he loves to play football.

4.5.2 Disjunction

A proposition “ $A \vee B$ ” with connective $\vee$ is known as *disjunction* of A and B. It is a proposition (or operation) which is true when at least one of the constituent propositions is true. The output is false only when both the input propositions are false. It is also referred to as OR-ing the propositions. Example:

I will go to her house or she will come to my house. It can be written as:

A = I will go to her house.

B = She will come to my house.

$A \vee B$ = I will go to her house or she will come to my house.

4.5.3 Negation

The proposition $\neg A$ (or $\sim A$) with $\neg$ (or $\sim$) connective is known as the *negation* of A. The purpose of negation is to negate the logic of a given proposition. If A is true, its negation will be false, and if A is false, its negation will be true. Example:

University is closed. It can be written as:

A = University is closed.

$\neg A$ = University is not closed.

4.5.4 Implication

The proposition $A \rightarrow B$ with the $\rightarrow$ connective is known as A *implies* B . It is also called *if-then* proposition. Here, the second proposition is a logical consequence of the first proposition. For example, “If Mary scores good in examinations, I will buy a mobile phone for her”. In this case, it means that if Mary scores good, she will definitely get the mobile phone but it doesn’t mean that if she performs bad, she won’t get the mobile phone. In set notation, we can also say that $A \subseteq B$ i.e., if something exists in the set A , then it necessarily exists in the set B . Another example:

If you score above 90%, you will get a mobile phone.

A = You score above 90%.

B = You will get a mobile phone.

$A \rightarrow B$ = If you score above 90%, you will get a mobile phone.

4.5.5 Bi-conditional

A proposition $A \leftrightarrow B$ with connective $\leftrightarrow$ is known as a *biconditional* or *if-and-only-if* proposition. It is true when both the atomic propositions are true or both are false. A classic example of biconditional is “A triangle is equilateral if and only if all its angles are 60° each”. This statement means that if a triangle is an equivalent triangle, then all of its angles are 60° each. There is one more associated meaning with this statement, which means that if all the interior angles of a triangle are of 60° each, then it's an equivalent triangle. Example:

You will succeed in life if and only if you work hard.

A = You will succeed in life.

B = You work hard.

$A \leftrightarrow B$ = You will succeed in life if and only if you work hard.

☞ Check Your Progress 2

Which of the following propositions are atomic and which are compound?

1. The first battle of Panipat was fought in 1556.

.....
.....

2. Jack either plays cricket or football.

.....
.....

3. Posthumously, at the age of 22, Neerja Bhanot became the youngest recipient of the Ashok Chakra award which is India's highest peacetime gallantry decoration.

-
-
4. Chandigarh is the capital of the Indian states Haryana and Punjab.
-
-
5. Earth takes 365 days, 5 hours, 59 minutes and 16 seconds to complete one revolution around the Sun.
-
-
6. Dermatology is the branch of medical science which deals with the skin.
-
-
7. Indian sportspersons won 7 medals at the 2020 Summer Olympics and 19 medals at the 2020 Summer Paralympics both held at the Japanese city of Tokyo.
-
-
8. Harappan civilization is considered to be the oldest human civilization and it lies in the parts of present-day India, Pakistan and Afghanistan.
-
-
9. IGNOU is a central university and offers courses through the distance learning mode.
-
-
10. Uttarakhand was carved out of the Indian state of Uttar Pradesh in the year 2000.
-
-

4.6 SEMANTICS

Many of you had already learned various of the concepts to be covered in this unit, in Discrete Mathematics. If not, refer to MCS-212. Here is a quick revision to those concepts, and we will extend our discussion to the advanced concepts, which are useful for our field of work, i.e., Artificial Intelligence. **Propositions are** declarative sentences or statements that are either true or false, but not both; such sentences can either be **universally true** or **universally false**.

On the other hand, consider the declarative sentence ‘Women are more intelligent than men’. Some people may think it is true, while others may disagree. So, it is neither universally true nor universally false. Such a sentence is not acceptable as a statement or proposition in mathematical logic.

Note that a **proposition should be either uniformly true or uniformly false**.

In propositional logic, as mentioned earlier, symbols are also used to denote propositions. For instance, we may denote the propositions discussed above as follows:

P : The sun rises in the west,

Q : Sugar is sweet,

R : Ram has a Ph.D. degree.

The symbols, such as P, Q, and R, that are used to denote propositions, are called **atomic formulas, or atoms**. In this case, the truth-value of P is False, the truth-value of Q is True and the truth-value of R, though not known yet, is exactly one of ‘True’ or ‘False’, depending on whether Ram is actually a Ph. D or not.

At this stage, it may be noted that once symbols are used in place of given statements in, say, English, then the propositional system, and, in general, a symbolic system, is aware **only** of symbolic representations and the associated truth values. The system operates only on these representations. And, except for possible final translation, is **not aware** of the original statements, generally given in some natural language, say, English.

When you’re talking to someone, do you use very simple sentences only? Don’t you use more complicated ones which are joined by words like ‘and’, ‘or’, etc? In the same way, most statements in mathematical logic are combinations of simpler statements joined by words and phrases like ‘and’, ‘or’, ‘if ... then’. ‘If and only if’, etc. We can build, from atoms, *more complex propositions*, sometimes called compound **propositions**, by using logical **connectives**,

The Logical Connectives are used to frame compound propositions, and they are as follows:

a) Disjunction: The **disjunction** of two propositions p and q is the compound statement **p or q**, denoted by $p \vee q$.

The **exclusive disjunction** of two propositions p and q is the statement **‘Either of the two (i.e. p or q) can be true, but both can’t be true’**. We denote this by $p \oplus q$.

b) Conjunction: We call the compound statement **‘p and q’** the **conjunction** of the statements p and q. We denote this by $p \wedge q$.

c) Negation: The **negation** of a proposition p is **‘not p’**, denoted by $\sim p$.

d) Implication (Conditional Connectives) Given any two propositions p and q, we denote the statement **‘If p, then q’** by $p \rightarrow q$. We also read this as ‘p implies q’. or ‘p is sufficient for q’, or ‘p **only if** q’. We also call p the

hypothesis and q the conclusion. Further, a statement of the form $p \rightarrow q$ is called a **conditional statement** or a **conditional proposition**.

Let p and q be two propositions. The compound statement $(p \rightarrow q) \wedge (q \rightarrow p)$ is the **bi-conditional** of p and q . We denote it by $p \leftrightarrow q$, and read it as ‘ p **if and only** q ’

Note: The two connectives $\rightarrow$ and $\leftrightarrow$ are called **conditional connectives**

The rule of precedence: The order of preference in which the connectives are applied in a formula of propositions that has no brackets is as follows:

- i) $\sim$
- ii) $\wedge$
- iii) $\vee$ and $\oplus$
- iv) $\rightarrow$ and $\leftrightarrow$

Note that the ‘inclusive or’ and ‘exclusive or’ are both third in the order of preference. However, if both these appear in a statement, we first apply the left most one. So, for instance, in $p \vee q \oplus \sim p$, we first apply $\vee$ and then $\oplus$. The same applies to the ‘implication’ and the ‘biconditional’, which are both fourth in the order of preference.

Let’s see the working of the various concepts learned above, with the help of truth tables. In the following truth table, we write every TRUE value as T and every FALSE value as F.

4.6.1 Negation Truth Table

α	$\sim \alpha$
F(0)	_T(1)_
T(1)	_F(0)_

4.6.2 Conjunction/Disjunction/Implication /Biconditional Truth Table

α_1	α_2	Conjunction $\alpha_1 \wedge \alpha_2$	Disjunction $\alpha_1 \vee \alpha_2$	Implication $\alpha_1 \rightarrow \alpha_2$	Biconditional $\alpha_1 \leftrightarrow \alpha_2$
F(0)	_F(0)_	_F(0)_	_F(0)_	_T(1)_	_T(1)_
F(0)	_T(1)_	_F(0)_	_T(1)_	_T(1)_	_F(0)_
T(1)	_F(0)_	_F(0)_	_T(1)_	_F(0)_	_F(0)_
T(1)	_T(1)_	_T(1)_	_T(1)_	_T(1)_	_T(1)_

4.6.3 Conjunction and Disjunction with three variables

Predicate And
Propositional Logic

α_1	α_2	α_3	$(\alpha_1 \wedge \alpha_2)$	$(\alpha_2 \wedge \alpha_3)$	$(\alpha_1 \vee \alpha_2)$	$(\alpha_2 \vee \alpha_3)$	$(\alpha_1 \wedge \alpha_2) \wedge \alpha_3$ or $\alpha_1 \wedge (\alpha_2 \wedge \alpha_3)$	$(\alpha_1 \vee \alpha_2) \vee \alpha_3$ or $\alpha_1 \vee (\alpha_2 \vee \alpha_3)$
$\neg T(1)$	$\neg T(1)$	$\neg T(1)$	$\neg T(1)$	$\neg T(1)$	$\neg T(1)$	$\neg T(1)$	$\neg T(1)$	$\neg T(1)$
$\neg T(1)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$
$\neg T(1)$	$\neg F(0)$	$\neg T(1)$	$\neg F(0)$	$\neg F(0)$	$\neg T(1)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$
$\neg T(1)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg T(1)$	$\neg F(0)$	$\neg F(0)$	$\neg T(1)$
$\neg F(0)$	$\neg T(1)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$	$\neg T(1)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$
$\neg F(0)$	$\neg T(1)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg T(1)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$
$\neg F(0)$	$\neg F(0)$	$\neg T(1)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg T(1)$	$\neg F(0)$	$\neg T(1)$
$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$	$\neg F(0)$

Using these logical connectives, we can transform any sentence into its equivalent mathematical representation in Symbolic Logic, and that representation is referred to as Well Form Formula (WFF). Let's briefly discuss it here also, as it has wide applications in Artificial Intelligence.

A Well-formed formula, or *wff*, or *formula* in short, in propositional logic is defined recursively as follows:

1. An atom is a wff.
2. If A is a wff, then $(\neg A)$ is a wff.
3. If A and B are wffs, then each of $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, and $(A \leftrightarrow B)$ is a wff.
4. Any wff is obtained only by applying the above rules.

From the above recursive definition of a wff it is not difficult to see that expression:

$((P \rightarrow (Q \wedge (\neg R))) \leftrightarrow S)$ is a wff; because, to begin with, each of P, Q, $(\neg R)$ and S, by definitions is a wff. Then, by recursive application, the expression: $(Q \wedge (\neg R))$ is a wff. Again, by another recursive application, the expression: $(P \rightarrow (Q \wedge (\neg R)))$ is a wff, and, finally, the expression given initially is a wff.

Further, it is easy to see that according to the recursive definition of a wff, each of the expressions: $(P \rightarrow (Q \wedge))$ and $(P (Q \wedge R))$ is **not** a wff.

Some pairs of parentheses may be dropped for simplification. For example,

$A \vee B$ and $A \rightarrow B$, respectively, may be used instead of the given wffs $(A \vee B)$ and $(A \rightarrow B)$, respectively. We can omit the use of parentheses by assigning *priorities in increasing order* to the connectives as follows:

$\leftrightarrow, \rightarrow, \vee, \wedge, \neg$.

Thus, ' $\leftrightarrow$ ' has least priority and ' $\sim$ ' has highest priority. Further, if in an expression, there are no parentheses and two connectives between three atomic formulas are used, then the operator with higher priority will be applied first and the other operator will be applied later.

For example: Let us be given the wff $P \rightarrow Q \wedge \sim R$ without parentheses. Then among the operators appearing in wff, the operator ' $\sim$ ' has the highest priority. Therefore, $\sim R$ is replaced by $(\sim R)$. The equivalent expression becomes $P \rightarrow Q \wedge (\sim R)$. Next, out of the two operators, viz. ' $\rightarrow$ ' and ' $\wedge$ ', the operator ' $\wedge$ ' has higher priority. Therefore, by applying parentheses appropriately, the new expression becomes $P \rightarrow (Q \wedge (\sim R))$. Finally, only one operator is left. Hence the fully parenthesized expression becomes $(P \rightarrow (Q \wedge (\sim R)))$

The following are the rules of finding the truth value or meaning of a wff, when the truth values of the atoms appearing in the wff are known or given.

1. The wff $\sim A$ is *True* when A is *False*, and $\sim A$ is *False* when A is *true*. The wff $\sim A$ is called the **negation** of A .
2. The wff $(A \wedge B)$ is *True* if A and B are both *True*; otherwise, the wff $A \wedge B$ is *False*. The wff $(A \wedge B)$ is called the **conjunction** of A and B .
3. The wff $(A \vee B)$ is *true* if at least one of A and B is *True*; otherwise, $(A \vee B)$ is *False*. $(A \vee B)$ is called the **disjunction** of A and B .
4. The wff $(A \rightarrow B)$ is *False* if A is *True* and B is *False*; otherwise, $(A \rightarrow B)$ is *True*. The wff $(A \rightarrow B)$ is read as "If A , then B ," or " **A implies B** ." The symbol ' $\rightarrow$ ' is called **implication**.
5. The wff $(A \leftrightarrow B)$ is *True* whenever A and B have the same truth values; otherwise $(A \leftrightarrow B)$ is *False*. The wff $(A \leftrightarrow B)$ is read as " **A if and only if B** ."

☞ Check Your Progress 3

1. Draw the truth table for the following:
 - a) $\alpha_2 \leftrightarrow (\sim \alpha_1 \rightarrow (\alpha_1 \vee \alpha_2))$
 - b) $(\sim \alpha_1 \leftrightarrow (\alpha_2 \leftrightarrow \alpha_3)) \vee (\alpha_3 \wedge \alpha_2)$
 - c) $((\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3) \vee \sim \alpha_4$
 - d) $((\alpha_1 \rightarrow \sim \alpha_2) \leftrightarrow \alpha_3) \rightarrow \sim (\alpha_1 \vee \alpha_1)$

.....
.....

2. Verify the De Morgan's Laws using Truth Tables

.....
.....

3. Write WFF for the following statements:

- a) Every Person has Mother

4.7 PROPOSITIONAL RULES OF INFERENCE

We need intelligent computers based on the concept of artificial intelligence which are able to infer new “knowledge” or logic from the existing logic using the theory of inference. Inference rules help us to infer new propositions and conclusions based on existing propositions and logic. They act as templates to generate new arguments from the premises or predicates. We deduce new statements from the statements whose truthfulness is already known. These rules come to the rescue when we need to prove something logically. In general, inference rules preserve the truth. Depending on the problem, some or all of these rules may be applied to infer new propositions. The procedure for determining whether a proposition is a conclusion of the given propositions is known as inferring a proposition. The inference rules are described below.

4.7.1 Modus Ponens (MP)

It states that if the propositions $A \rightarrow B$ and A are true, then B is also true. Modus Ponens is also referred to as the implication elimination because it eliminates the implication $A \rightarrow B$ and results in only the proposition B . It also affirms the truthfulness of the antecedent. It is written as:

$$\alpha_1 \rightarrow \alpha_2, \alpha_1 \Rightarrow \alpha_2$$

4.7.2 Modus Tollens (MT)

It states that if $A \rightarrow B$ and $\neg B$ are true then $\neg A$ is also true. Modus Tollens is also referred to as denying the consequent, as it denies the truthfulness of the consequent. The rule is expressed as:

$$\alpha_1 \rightarrow \alpha_2, \sim \alpha_2 \Rightarrow \sim \alpha_1$$

4.7.3 Disjunctive Syllogism (DS)

Disjunctive Syllogism affirms the truthfulness of the other proposition if one of the propositions in a disjunction is false.

Rule 1: $\alpha_1 \vee \alpha_2, \sim \alpha_1 \Rightarrow \alpha_2$

Rule 2: $\alpha_1 \vee \alpha_2, \sim \alpha_2 \Rightarrow \alpha_1$

4.7.4 Addition

The rule states that if a proposition is true, then its disjunction with any other proposition is also true.

Rule 1: $\alpha_1 \Rightarrow \alpha_1 \vee \alpha_2$

Rule 2: $\alpha_2 \Rightarrow \alpha_1 \vee \alpha_2$

4.7.5 Simplification

Simplification means that if we have a conjunction, then both the constituent propositions are also true.

Rule 1: $\alpha_1 \wedge \alpha_2 \Rightarrow \alpha_1$

Rule 2: $\alpha_1 \wedge \alpha_2 \Rightarrow \alpha_2$

4.7.6 Conjunction

A conjunction states that if two propositions are true, then their conjunction is also true. It is written as:

$$\alpha_1, \alpha_2 \Rightarrow \alpha_1 \wedge \alpha_2$$

4.7.7 Hypothetical Syllogism (HS)

The rule says that the conclusion $\alpha_1 \rightarrow \alpha_3$ is true whenever conditional statements $\alpha_1 \rightarrow \alpha_2$ and $\alpha_2 \rightarrow \alpha_3$ hold the truth values. These rules also show the transitive nature of the implication operator.

$$\alpha_1 \rightarrow \alpha_2, \alpha_2 \rightarrow \alpha_3 \Rightarrow \alpha_1 \rightarrow \alpha_3$$

4.7.8 Absorption

The rule states that if the literal α_1 conditionally implies another literal α_2 i.e., $\alpha_1 \rightarrow \alpha_2$ is true, then $\alpha_1 \rightarrow (\alpha_1 \wedge \alpha_2)$ also holds.

$$\alpha_1 \rightarrow \alpha_2 \Rightarrow \alpha_1 \rightarrow (\alpha_1 \wedge \alpha_2)$$

4.7.9 Constructive Dilemma (CD)

According to the rule, if proposition $(\alpha_1 \vee \alpha_3)$ and proposition $((\alpha_1 \rightarrow \alpha_2) \wedge (\alpha_3 \rightarrow \alpha_4))$ have true values, then the well-formed formula $(\alpha_2 \vee \alpha_4)$ also holds a true value.

$$(\alpha_1 \vee \alpha_3), (\alpha_1 \rightarrow \alpha_2) \wedge (\alpha_3 \rightarrow \alpha_4) \Rightarrow \alpha_2 \vee \alpha_4$$

4.8 PROPOSITIONAL RULES OF REPLACEMENT

A proposition is a specialized statement whereas a predicate is a generalized statement". To be more specific, the propositions use the logical connectives only, and the predicates use logical connectives and quantifiers (universal and existential), both.

Note: $\exists$ is the symbol used for the Existential quantifier, and $\forall$ is used for the Universal quantifier.

In predicate logic, a replacement rule is used to replace an argument or a set of arguments with an equivalent argument. By equivalent arguments, we mean that the logical interpretation of the arguments is the same. These rules are used to manipulate the propositions. Also, the axioms and the propositional rules of inference are used as an aid to generate the replacement rules. Given

below is the table summarizing the different replacement rules over the propositions α_1 , α_2 and α_3 .

Replacement Rule	Proposition	Equivalent
Tautology (Conjunction of a statement with itself always implies the statement)	$\alpha_1 \wedge \alpha_1$	α_1
Double Negation (DN) (Also called negation of negation)	$(\sim(\sim\alpha_1))$	α_1
Commutativity (Valid for Conjunction and Disjunction)	$\alpha_1 \wedge \alpha_2$	$\alpha_2 \wedge \alpha_1$
	$\alpha_1 \vee \alpha_2$	$\alpha_1 \vee \alpha_2$
Associativity (Valid for Conjunction and Disjunction)	$(\alpha_1 \wedge \alpha_2) \wedge \alpha_3$	$\alpha_1 \wedge (\alpha_2 \wedge \alpha_3)$
	$(\alpha_1 \vee \alpha_2) \vee \alpha_3$	$\alpha_1 \vee (\alpha_2 \vee \alpha_3)$
DeMorgan's Laws	$\sim(\alpha_1 \wedge \alpha_2)$	$(\sim\alpha_1) \vee (\sim\alpha_2)$
	$\sim(\alpha_1 \vee \alpha_2)$	$(\sim\alpha_1) \wedge (\sim\alpha_2)$
Transposition (Defined over implication)	$\alpha_1 \rightarrow \alpha_2$	$(\sim\alpha_2) \rightarrow (\sim\alpha_1)$
Exportation	$\alpha_1 \rightarrow (\alpha_2 \rightarrow \alpha_3)$	$(\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3$
Distribution (AND over OR, and OR over AND)	$\alpha_1 \wedge (\alpha_2 \vee \alpha_3)$	$(\alpha_1 \wedge \alpha_2) \vee (\alpha_1 \wedge \alpha_3)$
	$\alpha_1 \vee (\alpha_2 \wedge \alpha_3)$	$(\alpha_1 \vee \alpha_2) \wedge (\alpha_1 \vee \alpha_3)$
Material Implication (MI) (Defined over implication)	$\alpha_1 \rightarrow \alpha_2$	$\sim\alpha_1 \vee \alpha_2$
Material Equivalence (ME)(Defined over biconditional)	$\alpha_1 \leftrightarrow \alpha_2$	$(\alpha_1 \rightarrow \alpha_2) \wedge (\alpha_2 \rightarrow \alpha_1)$

4.9 VALIDITY AND SATISFIABILITY

An argument is called *valid* in propositional logic if the argument is a tautology for all possible combinations of the given premises. An argument is called *satisfiable* if the argument is true for at least one combination of the premises. Consider the example given below. Kindly note, every TRUE value is written as $_T(1)_$, and every FALSE value is written as $_F(0)_$.

Example 1: Consider the expression $\alpha_1 \vee (\alpha_2 \vee \alpha_3) \vee (\sim\alpha_2 \wedge \sim\alpha_3)$, whose truth table is given below.

α_1	α_2	α_3	$\sim\alpha_2$	$\sim\alpha_3$	$\alpha_1 \vee \alpha_2$	$\sim\alpha_2 \wedge \sim\alpha_3$	$\alpha_1 \vee (\alpha_2 \vee \alpha_3) \vee (\sim\alpha_2 \wedge \sim\alpha_3)$
F(0)	F(0)	F(0)	T(1)	T(1)	F(0)	T(1)	T(1)
F(0)	F(0)	T(1)	T(1)	F(0)	T(1)	F(0)	T(1)
F(0)	T(1)	F(0)	F(0)	T(1)	T(1)	F(0)	T(1)
F(0)	T(1)	T(1)	F(0)	F(0)	T(1)	F(0)	T(1)
T(1)	F(0)	F(0)	T(1)	T(1)	F(0)	T(1)	T(1)
T(1)	F(0)	T(1)	T(1)	F(0)	T(1)	F(0)	T(1)
T(1)	T(1)	F(0)	F(0)	T(1)	T(1)	F(0)	T(1)
T(1)	T(1)	T(1)	F(0)	F(0)	T(1)	F(0)	T(1)

It is noteworthy from the above truth table that the argument $\alpha_1 \vee (\alpha_2 \vee \alpha_3) \vee (\sim\alpha_2 \wedge \sim\alpha_3)$ is a valid argument as it has true values for all possible combinations of the premises α_1 , α_2 and α_3 .

Example 2: Consider the expression $\alpha_1 \wedge ((\alpha_2 \wedge \alpha_3) \vee (\alpha_1 \wedge \alpha_3))$ for the

α_1	α_2	α_3	$\alpha_1 \wedge \alpha_3$	$\alpha_2 \wedge \alpha_3$	$(\alpha_2 \wedge \alpha_3) \vee (\alpha_1 \wedge \alpha_3)$	$\alpha_1 \wedge ((\alpha_2 \wedge \alpha_3) \vee (\alpha_1 \wedge \alpha_3))$
F(0)	F(0)	F(0)	F(0)	F(0)	F(0)	F(0)
F(0)	F(0)	T(1)	T(1)	F(0)	T(1)	F(0)
F(0)	T(1)	F(0)	F(0)	F(0)	F(0)	F(0)
F(0)	T(1)	T(1)	T(1)	T(1)	T(1)	F(0)
T(1)	F(0)	F(0)	T(1)	F(0)	T(1)	T(1)
T(1)	F(0)	T(1)	T(1)	F(0)	T(1)	T(1)
T(1)	T(1)	F(0)	T(1)	F(0)	T(1)	T(1)
T(1)	T(1)	T(1)	T(1)	T(1)	T(1)	T(1)

propositions α_1 , α_2 , and α_3 whose truth table is given below.

In this example, as the argument $\alpha_1 \wedge ((\alpha_2 \wedge \alpha_3) \vee (\alpha_1 \wedge \alpha_3))$ is true for a few combinations of the premises α_1 , α_2 , and α_3 , hence it is a satisfiable argument.

4.10 INTRODUCTION TO PREDICATE LOGIC

Now it's time to understand the difference between the Proposition and the predicate (also known as a propositional function). In short, a proposition is a specialized statement whereas Predicate is a generalized statement. To be more specific, the propositions use the logical connectives only, and the predicates use logical connectives and quantifiers (universal and existential), both.

Note: $\exists$ is the symbol used for the Existential quantifier, and $\forall$ is used for the Universal quantifier.

Let's understand the difference through some more details, as given below.

In a propositional function, or a predicate, a variable x in a sentence $p(x)$ becomes a proposition when we give x a definite value from the set of values it

can take. We usually denote such functions by $p(x)$, $q(x)$, etc. The set of values x can take is called the universe of discourse.

So, if $p(x)$ is ' $x > 5$ ', then $p(x)$ is not a proposition. But when we give x particular values, say $x = 6$ or $x = 0$, then we get propositions. Here, $p(6)$ is a true proposition and $p(0)$ is a false proposition.

Similarly, if $q(x)$ is ' x has gone to Patna,' then replacing x by 'Taj Mahal' gives us a false proposition.

Note that a predicate is usually not a proposition. But, of course, every proposition is a propositional function in the same way that every real number is a real-valued function, namely, the constant function.

Now, can all sentences be written in symbolic form by using only the logical connectives? What about sentences like ' x is prime and $x + 1$ is prime for some x .'? How would you symbolize the phrase 'for some x ', which we can rephrase as 'there exists an x '? You must have come across this term often while studying mathematics. **We use the symbol ' $\exists$ ' to denote this quantifier, 'there exists'.** The way we use it is, for instance, to rewrite 'There is at least one child in the class.' As ' $(\exists x \text{ in } U)p(x)$ ', where $p(x)$ is the sentence ' x is in the class.' and U is the set of all children.

Now, suppose we take the negative of the proposition we have just stated. Wouldn't it be 'There is no child in the class.'? We could symbolize this as 'for all x in U , $q(x)$ ' where x ranges over all children and $q(x)$ denotes the sentence ' x is not in the class.', i.e., $q(x) \equiv \sim p(x)$.

We have a **mathematical symbol for the quantifier 'for all', which is ' $\forall$ '.** So, the proposition above can be written as:

' $(\forall x \in U)q(x)$ ', or ' $q(x), \forall x \in U$ '.

An example of the use of the existential quantifier is the true statement.

$(\exists x \in \mathbf{R})(x + 1 > 0)$, which is read as 'There exists an x in $\mathbf{R}$ for which $x + 1 > 0$.'

Another example is the false statement

$(\exists x \in \mathbf{N})(x - \frac{1}{2} = 0)$, which is read as 'There exists an x in $\mathbf{N}$ for which $x - \frac{1}{2} = 0$.'

An example of the use of the universal quantifier is $(\forall x \notin \mathbf{N})(x^2 > x)$, which is read as 'for every x not in $\mathbf{N}$, $x^2 > x$ '. Of course, this is a false statement, because there is at least one $x \notin \mathbf{N}$, $x \in \mathbf{R}$, for which it is false.

As you have already read in the example of a child in the class,

$(\forall x \in U)p(x)$ is logically equivalent to $\sim (\exists x \in U)(\sim p(x))$. Therefore,

$\sim (\forall x \in U)p(x) \equiv \sim \sim (\exists x \in U)(\sim p(x)) \equiv (\exists x \in U)(\sim p(x))$.

This is one of the rules for negation that relate $\forall$ and $\exists$. The two rules are

$$\sim (\forall x \in U)p(x) \equiv (\exists x \in U) (\sim p(x)), \text{ and}$$

$$\sim (\exists x \in U)p(x) \equiv (\forall x \in U) (\sim p(x))$$

Where U is the set of values that x can take.

To Sum up “**a proposition is a specialized statement whereas Predicate is a generalized statement**”. To be more specific, the propositions use the logical connectives only, and the predicates use logical connectives and quantifiers (universal and existential), both.

Note: $\exists$ is the symbol used for the Existential quantifier and $\forall$ is used for the Universal quantifier.

By the interpretation of symbolic logic, we mean assigning a meaning to the symbols of any formal language. By default, the statements do not have any meaning associated with them, but when we assign the symbols to them, their meaning is automatically associated with them, i.e., TRUE (T) or FALSE (F).

Example 1: Consider the following statements

- a) Weather is not hot today, and it is windy than yesterday.
- b) Kids will go to park only if it is not hot.
- c) If kids do not go to park, then they can play near pool.
- d) If kids play near pool, then we can offer them a juice.

Let the statements be represented using literals as following:

H: Weather is not hot today

W: It is windy than yesterday

K: Kids will go to park

P: They can play near pool.

J: We can offer them juice.

The given statements can be interpreted as:

- a) $H \wedge W$
- b) $K \rightarrow \sim H$
- c) $\sim K \rightarrow P$
- d) $P \rightarrow J$

These interpretations may be verified with the help of truth tables.

4.11 INFERENCE IN PREDICATE LOGIC

In general, we are given a set of arguments in predicate logic. Now, using the rules of inference, we can deduce other arguments (or predicates) based on the given arguments (or predicates). This process is known as entailment as we entail new arguments (or predicates). The inference rules you learned in section 4.7 above, of this unit are also applicable here for the process of entailment or making inferences. Now, with the help of the following example, we will learn how the rules of inference, discussed above, can be used to solve the problems.

Example: There is a village that consists of two types of people – those who always tell the truth, and those who always lie. Suppose that you visit the village and two villagers A and B come up to you. Further, suppose

A says, “B always tells the truth,” and

B says, “A and I are of opposite types”.

What types are A and B ?

Solution: Let us start by assuming A is a truth-teller.

- ∴ What A says is true.
- ∴ B is a truth-teller.
- ∴ What B says is true.
- ∴ A and B are of opposite types.

This is a contradiction, because our premises say that A and B are both truth-tellers.

- ∴ The assumption we started with is false.
- ∴ A always tells lies.
- ∴ What A has told you is lie.
- ∴ B always tells lies.
- ∴ A and B are of the same type, i.e., both of them always lie.

Let us now consider the problem of showing that a statement is false, i.e., by using counterexamples: A common situation in which we look for counterexamples is to disprove statements of the form $p \rightarrow q$ needs to be an example where $p \wedge \sim q$. Therefore, a counterexample to $p \rightarrow q$ needs to be an example where $p \wedge \sim q$ is true, i.e., p is true, and $\sim q$ is true, i.e., the hypothesis p holds, but the conclusion q does not hold.

For instance, to disprove the statement ‘If n is an odd integer, then n is prime,’ we need to look for an odd integer that is not a prime number. 15 is one such integer. So, $n = 15$ is a counterexample to the given statement.

Notice that a **counter example to a statement p proves that p is false, i.e., $\sim p$ is true.**

Example: The following statements are to be symbolized and thereafter construct a proof for the following valid argument:

- (i) If the BOOK_X is literally true, then the Earth was made in six days.
- (ii) If the Earth was made in six days, then carbon dating is useless and Scientists/Researchers are liars.
- (iii) Scientists/Researchers are not liars.
- (iv) The BOOK_X is literally true, hence
- (v) God does not exist.

Solution: Let us symbolize as follows:

B: BOOK_X is literally true

E: The Earth was created in six days

C: Carbon dating techniques are useless

S: Scientists/Researchers are frauds

G: God exists

Therefore, the statements in the given arguments are symbolically represented as :

(i) $B \rightarrow E$

(ii) $E \rightarrow C \wedge S$

(iii) $\sim S$

(iv) B

(v) $\sim G$ (to show)

Using Modus Ponens on (i) and (iv), we get expression (vi) E

Using Modus Ponens on (ii) & (vi), we get expression (vii) $C \wedge S$

Using Simplification on (vii), we get expression (viii) S

Using Addition on (viii), we get expression (ix) $S \vee \sim G$

Using Disjunctive Syllogism(D.S.) on (iii) & (ix) we get expression (x) $\sim G$

The last statement is what is to be proved.

Remarks: (iii) and (viii) contradict each other in the above deduction. In general, if we come across two statements (like S and $\sim S$) that contradict each other during the process of deduction, we can deduce any statement, even if the statement can never be True in any way. So, we can assume that any statement is true if both S and $\sim S$ have already happened in the process of derivation.

4.12 PROOF SYSTEMS

A sequence of statements that follows logically from the previous set of statements or observations is a mathematical proof in propositional logic. The last statements in the proof become the theorem. This proof system symbolizes the science of valid inference.

There are more than two main styles of proof systems for propositional logic, but the two main styles are:

a) *Axiomatic Proof Systems and*

b) *Natural deduction Systems*

Axiomatic Proof Systems: This is a system where a conclusion is derived from either the given hypothesis or by using assumed premises, which are considered as truth. Such a hypothesis or assumed premise is an *axiom*.

Example 1: Show that $(\alpha_1 \vee \alpha_2)$ is a logical consequence of $(\alpha_2 \wedge \alpha_1)$ using proof systems.

Proof:

Step 1: $(\alpha_2 \wedge \alpha_1)$ (Premise)

Step 2: α_1 (Simplification, Step 1)

Step 3: α_2 (Simplification, Step 1)

Step 4: $\alpha_1 \vee \alpha_2$ (Addition on either Step 2 or Step 3)

We observe here that each step is either a truth or a logical consequence of previously established truths. In general, there can be more than one proof for establishing a given conclusion.

Thus, the proof system in propositional logic is quite similar to the proofs in mathematics, which follow a step-wise derivation of the consequent from the given hypothesis. However, in propositional logic, we use well-formed formulas obtained from literals and connectors, rather than using English statements. We use the rules of inference (Section 4.7) and replacement rules (Section 4.8) to prove our conclusion.

Example 2: Show that $\alpha_1 \rightarrow \sim \alpha_4$ can be derived from the given premises:

$$(\alpha_1 \rightarrow (\alpha_2 \vee \alpha_3)), \alpha_2 \rightarrow \sim \alpha_1, \alpha_4 \rightarrow \sim \alpha_3$$

Proof:

Step 1: $(\alpha_1 \rightarrow (\alpha_2 \vee \alpha_3))$ (Premise)

Step 2: α_1 (Assumed Premise)

Step 3: $\alpha_2 \vee \alpha_3$ (Modus Ponens, Step 1, 2)

Step 4: $\alpha_2 \rightarrow \sim \alpha_1$ (Premise)

Step 5: $\sim \alpha_2$ (Modus Tollens, Step 2, step 4)

Step 6: α_3 (Disjunctive Syllogism, Step 1, 2, 4)

Step 7: $\alpha_4 \rightarrow \sim \alpha_3$ (Premise)

Step 8: $\sim \alpha_4$ (Modus Tollens, Step 1, 2, 4, 7)

Step 9: $\alpha_1 \rightarrow \sim \alpha_4$ (Step 2, 4)

Hence proved.

The discussion over Natural Deduction Systems is given in section 4.13 below

4.13 NATURAL DEDUCTIONS

So far, we have discussed methods of solving problems requiring reasoning of propositional logic, which were based on

- i) Truth-table construction
- ii) Use of inference rules,

and follow, directly or indirectly, **the natural deduction approach.**

To determine whether or not a conclusion C in an argument is valid or invalid based on a given set of facts or axioms $A_1, A_2, \dots, A_n$, the only thing we

currently know is that either a truth table should be constructed for the formula $P: A_1 \wedge A_2 \wedge \dots \wedge A_n \rightarrow C$, or this formula should be converted to CNF or DNF by substituting equivalent formulas and simplifying it. There are other possible options available as well. The trouble with these methods, on the other hand, is that as the number n of axioms increases, the formula becomes more complicated (imagine n being equal to 50), and the number of variables involved, say k , also normally increases. When there are k different variables to consider in an argument, the size of the truth table grows to 2^k . For big values of k , the number of rows, denoted by 2^k , approaches an almost unmanageable level. As a result, it is necessary for us to look for alternative approaches that, rather than processing the entire argument as a single formula, process each of the individual formulas A_1 , A_2 , and C of the argument as well as their derivatives by applying some principles that ensure the validity of the results.

In an earlier section, we introduced eight different inference rules that can be used in propositional logic to help derive logical inferences. The methods of drawing valid conclusions that have been discussed up until this point are examples of an approach to drawing valid conclusions that is called the **natural deduction approach** of making inferences. This is an approach to drawing valid conclusions in which the reasoning system starts the reasoning process from the axioms, uses inference rules, and, if the conclusion can be validly drawn, then it ultimately reaches the conclusion that was intended. On the other hand, there is a method of obtaining legitimate conclusions that is known as the **Refutation approach**. This method will be covered in the following section.

The normal forms (CNF and DNF) also play a vital role in both Natural deductions and Resolution approach. To understand the normal forms, we need to start with the basic concepts of clauses, literals etc.

Some Definitions: A **clause** is a disjunction of literals. For example, $(E \vee \sim F \vee \sim G)$ is a clause. But $(E \vee \sim F \wedge \sim G)$ is not a clause. A **literal** is either an atom, say A , or its negation, say $\sim A$.

Definition: A formula E is said to be in a **Conjunctive Normal Form (CNF)** if and only if E has the form $E: E_1 \wedge \dots \wedge E_n$, $n \geq 1$, where each of $E_1, \dots, E_n$ is a **disjunction** of literals.

Definition: A formula E is said to be in **Disjunctive Normal Form (DNF)** if and only if E has the form $E: E_1 \vee E_2 \vee \dots \vee E_n$, where each E_i is a **conjunction** of literals.

Examples: Let A , B and C be atoms. Then $F: (\sim A \wedge B) \vee (A \wedge \sim B \wedge \sim C)$ is a formula in a disjunctive normal form.

Example: Again $G: (\sim A \vee B) \wedge (A \vee \sim B \vee \sim C)$ is a formula in Conjunctive Normal Form, because it is a conjunction of the two disjunctions of literals viz of $(\sim A \vee B)$ and $(A \vee \sim B \vee \sim C)$

Example: Each of the following is neither in CNF nor in DNF

- (i) $(\sim A \vee B) \vee (A \wedge \sim B \vee C)$
- (ii) $(A \rightarrow B) \wedge (\sim B \wedge \sim A)$

Using table of equivalent formulas given above, any valid propositional logic formula can be transformed into CNF as well as DNF.

The steps for conversion to DNF are as follows

Step 1: Use the equivalences to remove the logical operators ' $\leftrightarrow$ ' and ' $\rightarrow$ ':

$$(i) E \leftrightarrow G = (E \rightarrow G) \wedge (G \rightarrow E)$$

$$(ii) E \rightarrow G = \sim E \vee G$$

Step 2: Remove $\sim$'s, if occur consecutively more than once, using

$$(iii) \sim(\sim E) = E$$

(iv) Use De Morgan's laws to take ' $\sim$ ' nearest to atoms

$$(v) \sim(E \vee G) = \sim E \wedge \sim G$$

$$(vi) \sim(E \wedge G) = \sim E \vee \sim G$$

Step 3: Use the distributive laws repeatedly

$$(vii) E \vee (G \wedge H) = (E \vee G) \wedge (E \vee H)$$

$$(viii) E \wedge (G \vee H) = (E \wedge G) \vee (E \wedge H)$$

Example: Obtain a disjunctive normal form for the formula $\sim (A \rightarrow (\sim B \wedge C))$.

$$\text{Consider } A \rightarrow (\sim B \wedge C) = \sim A \vee (\sim B \wedge C) \quad (\text{Using } (E \rightarrow F) = (\sim E \vee F))$$

$$\text{Hence, } \sim(A \rightarrow (\sim B \wedge C)) = \sim(\sim A \vee (\sim B \wedge C))$$

$$= \sim(\sim A) \wedge \sim(\sim B \wedge C) \quad (\text{Using } \sim(E \vee F) = \sim E \wedge \sim F)$$

$$= A \wedge (B \vee \sim C) \quad (\text{Using } \sim(\sim E) = E \text{ and } \sim(E \wedge F) = \sim E \vee \sim F)$$

$$= (A \wedge B) \vee (A \wedge \sim C) \quad (\text{Using } E \wedge (F \vee G) = (E \wedge F) \vee (E \wedge G))$$

However, if we are to obtain CNF of $(\sim A \rightarrow (\sim B \wedge C))$, in the last but one step, we obtain

$$\sim(A \rightarrow (\sim B \wedge C)) = A \wedge (B \vee \sim C), \text{ which is in CNF, because, each of } A \text{ and}$$

$(B \vee \sim C)$ is a disjunct.

Example: Obtain conjunctive Normal Form (CNF) for the formula:

$$D \rightarrow (A \rightarrow (B \wedge C))$$

Consider

$$D \rightarrow (A \rightarrow (B \wedge C)) \quad (\text{using } E \rightarrow F = \sim E \vee F \text{ for the inner implication})$$

$$= D \rightarrow (\sim A \vee (B \wedge C)) \quad (\text{using } E \rightarrow F = \sim E \vee F \text{ for the outer implication})$$

$$= \sim D \vee (\sim A \vee (B \wedge C))$$

$$= (\sim D \vee \sim A) \vee (B \wedge C) \text{ (using Associative law for disjunction)}$$

$$= ((\sim D \vee \sim A \vee B) \wedge (\sim D \vee \sim A \vee C))$$

The last line denotes the conjunctive Normal Form of $D \rightarrow (A \rightarrow (B \wedge C))$

(Using distributivity of $\vee$ over $\wedge$)

Note: If we stop at the last but one stop, then we obtain $(\sim D \vee \sim A) \vee (B \wedge C) = \sim D \vee \sim A \vee (B \wedge C)$ is a Disjunctive Normal Form for the given formula : $D \rightarrow (A \rightarrow (B \wedge C))$

Sum up of the technique of natural deduction - Apart from constructing truth table to show that the conclusion follows from the given set of premises, another method exists known as *natural deduction*. In this case, we assume that the conclusion is not valid. We consider negated conclusion as a premise along with other given premises. We apply certain implications, equivalences and replacement rules to derive a contradiction to our assumption. Once, we obtain a contradiction, this proves that the given argument is true.

Example 1: Show that Z is a valid conclusion from the premises X, $X \rightarrow Y$ and $Y \rightarrow Z$.

Proof:

- Step 1: $\sim Z$ (Negated conclusion as Premise)
- Step 2: X (Premise)
- Step 3: $X \rightarrow Y$ (Premise)
- Step 4: Y (using Step 2, 3 and Modus Ponens)
- Step 5: $Y \rightarrow Z$ (Premise)
- Step 6: Z (using Step 4, 5 and Modus Ponens)

We obtain the conclusion Z from the given premises, which is a contradiction to our assumption $\sim Z$. Thus, the given conclusion is valid.

Example 2: Show that $\sim P$ is concluded from RV S, $S \rightarrow \sim Q$, $P \rightarrow Q$ and $R \rightarrow \sim Q$.

Proof:

- Step 1: $\sim(\sim P)$ (Negated conclusion as Premise)
- Step 2: P (Step 1, Double negation)
- Step 3: $P \rightarrow Q$ (Premise)
- Step 4: Q (Step 2, 3, Modus Ponens)
- Step 5: $S \rightarrow \sim Q$ (Premise)
- Step 6: $\sim S$ (Step 4, 5, Modus Tollens)
- Step 7: $R \vee S$ (Premise)
- Step 8: R (Step 1, 3, 5, 7, Disjunctive Syllogism)
- Step 9: $R \rightarrow \sim Q$ (Premise)
- Step 10: $\sim Q$ (Step 1, 3, 5, 7, 9, and Modus Ponens)

We obtain $\sim Q$ in Step 10 and Q in step 4, which is a contradiction. Thus, our assumption is not valid. Hence, the conclusion follows from the set of premises.

4.14 PROPOSITIONAL RESOLUTION

For the most part, there are two distinct strategies that can be implemented in order to demonstrate the correctness of a theorem or derive a valid conclusion from a given collection of axioms:

- i) natural deduction
- ii) the method of refutation

In the method known as **natural deduction**, one begins with a given set of axioms, applies various rules of inference, and ultimately arrives at a conclusion. This method is strikingly similar to the intuitive reasoning that is characteristic of humans.

When using a **refutation approach**, one begins with the denial of the conclusion that is to be drawn and then proceeds to deduce a contradiction or the word "false." We are able to deduce a contradiction as a result of having presupposed that the conclusion is incorrect; hence, the premise that the conclusion is incorrect is itself incorrect. Therefore, the argument concerning the technique of resolution leads to the correctness of the conclusion. In this part of the article, we will talk about a different method known as the **Resolution Method**, which was proposed by Robinson in 1965 and is based on the **refutation approach**. The Robinson technique, which has served as the foundation for numerous computerised theorem provers, highlights the significance of the method in question.

Propositional resolution is a sound, complete and powerful rule of inference used in Propositional Logic. It is used to prove the unsatisfiability of the given set of statements. This is done using a strategy called Resolution Refutation that uses Resolution rule as described below.

Resolution Rule: The rule states that given two statements as $\{\alpha_1, \alpha_2, \alpha_3 \dots \alpha_m\}$ and $\{\gamma_1, \gamma_2, \gamma_3 \dots \gamma_n\}$, then the conclusion is $\{\alpha_1, \alpha_2, \alpha_3 \dots \alpha_m, \gamma_1, \gamma_2, \gamma_3, \dots \gamma_n\}$.

For example, the statements $\{A, B\}$ and $\{C, \sim B\}$ leads to the conclusion $\{A, C\}$.

Resolution Refutation:

- a) Convert all the given statements to Conjunctive Normal Form (CNF). It is also described as AND of ORs'. For eg., $(A \vee B) \wedge (\sim A \vee B) \wedge (\sim B \vee A)$ is a CNF.
- b) Obtain the negation of the given conclusion
- c) Apply the resolution rule until either the contradiction is obtained or the resolution rule cannot be applied anymore.

4.14.1 Clausal Form

Any atomic sentence or its negation is called a *literal*. A literal or the disjunction of at least two literals is called the *clausal form* or *clause expression*. Next, a *clause* is defined as the set of literals in the clause form or

clause expression. For example, consider two atomic statements represented using the literals X and Y. Their clausal expressions are X, $\sim X$ and $(X \vee Y)$ and the clauses for these expressions are $\{X\}$, $\{\sim X\}$ and $\{X, Y\}$. It is noteworthy that the empty set $\{\}$ is always a clause as it represents an empty disjunction and hence, is unsatisfiable. Kindly note, conjunctive normal form (CNF) of an expression represents the corresponding clausal form.

Now, we shall first understand certain rules for converting the statements to the clause form as given below.

- i) Operator: $(\beta_1 \vee \beta_2 \vee \beta_3 \vee \dots \vee \beta_m) \Rightarrow \{\beta_1, \beta_2, \beta_3, \dots, \beta_m\}$
 $(\beta_1 \wedge \beta_2 \wedge \beta_3 \wedge \dots \wedge \beta_m) \Rightarrow \{\beta_1\}, \{\beta_2\}, \{\beta_3\}, \dots, \{\beta_m\}$
- ii) Negation: same as Double Negation and De Morgan's Law in section 4.8
- iii) Distribution: as in sub-section 4.8
- iv) Implications: as Material implication and Material Equivalence (section 4.8)

Example 1: Convert $A \wedge (B \rightarrow C)$ to a clausal expression.

Step 1: $A \wedge (\sim B \vee C)$ (using rule Material Implication to eliminate $\rightarrow$)

Step 2: $\{A\}, \{\sim B \vee C\}$ (using rule Operator to eliminate $\wedge$)

Example 2: Derive the Clausal form or Conjunctive normal form of $X \leftrightarrow Y$.

Step 1: Replace the bi-condition using the Material equivalence rule:

$$(X \rightarrow Y) \wedge (Y \rightarrow X)$$

Step 2: Use the Material Implication replacement rule to replace the implication:

$$(\neg X \vee Y) \wedge (\neg Y \vee X)$$

Example 3: Derive the CNF of $\sim Z \wedge \sim((\sim X) \rightarrow (\sim Y))$

Step 1: Replace implication using Material Implication(MI):

$$\sim(\sim Z \wedge (\sim(\sim X) \vee \sim Y))$$

Step 2: Use double negation (DN):

$$\sim(\sim Z \wedge (X \vee \sim Y))$$

Step 3: Apply DeMorgan's Law:

$$\sim\sim Z \vee \sim(X \vee \sim Y)$$

Step 4: Apply Double Negation:

$$Z \vee \sim(X \vee \sim Y)$$

Step 5: Apply DeMorgan's Law again:

$$Z \vee (\sim X \wedge \sim\sim Y)$$

Step 6: Apply Double Negation on $\sim\sim Y$:

$$Z \vee (\sim X \wedge Y)$$

Step 7: Lastly, apply Distributive law to obtain CNF:

$(Z \vee \sim X) \wedge (Z \vee Y)$, which is the AND of OR's form.

$(Z \vee \sim X)$, $(Z \vee Y)$ are the clausal forms for the given expression.

4.14.2 Determining Unsatisfiability

If the obtained set of clauses is not satisfiable, then one can derive an empty clause using resolution principle as described above. In other words, to determine whether a set of propositions or premises $\{P\}$ logically entails a conclusion C , write $P \vee \{\neg C\}$ in clausal form and try to derive the empty clause as explained in examples below.

Example 1: Given a set of propositions : $X \rightarrow Y$, $Y \rightarrow Z$. Prove $X \rightarrow Z$.

Proof: To prove the conclusion, we add the negation of conclusion i.e $\sim(X \rightarrow Z)$ to the set of premises, and derive an empty clause.

Step 1: $X \rightarrow Y$ (Premise)

Step 2: $\sim X \vee Y$ (Premise, Material Implication)

Step 3: $Y \rightarrow Z$ (Premise)

Step 4: $\sim Y \vee Z$ (Premise, Material Implication)

Step 5: $\sim(X \rightarrow Z)$ (Premise, Negated Conclusion)

Step 6: $\sim(\sim X \vee Z)$ (Premise, Material Implication)

Step 7: $X \wedge \sim Z$ (Premise, DeMorgans)

Step 8: X (Clausal form, Operator)

Step 9: $\sim Z$ (Clausal form, Operator)

Step 10: Y (Resolution rule on Premises in Step 2 and Step 8)

Step 11: Z (Resolution rule on Step 10 and Step 4)

Step 12: $\{\}$ (Conjunction on Step 11 and Step 9)

Thus, the given set of premises entail the conclusion.

Example 2: Use propositional resolution to derive the goal from the given knowledge base.

- a) Either it is a head, or Lisa wins.
- b) If Lisa wins, then Mary will go.
- c) If it is a head, then the game is over.
- d) The game is not over.

Conclusion: Mary will go.

Proof: First consider propositions to represent each of the statement in knowledge base.

Let H: It is a head

L: Lisa wins

M: Mary will go

G: Game is over.

Re-writing the given knowledge base using the propositions defined.

- a) $H \vee L$
- b) $L \rightarrow M$
- c) $H \rightarrow G$
- d) $\sim G$

Conclusion: M

Step 1: $H \vee L$ (Premise)

Step 2: $L \rightarrow M$ (Premise)

Step 3: $\sim L \vee M$ (Step 2, Material Implication)

Step 4: $H \rightarrow G$ (Premise)

Step 5: $\sim H \vee G$ (Step 4, Material Implication)

Step 6: $\sim G$ (Premise)

Step 7: $\sim M$ (Negated conclusion as Premise)

Step 8: $H \vee M$ (Resolution principle on Step 1 and 3)

Step 9: $M \vee G$ (Resolution principle on Step 8 and 5)

Step 10: M (Resolution principle on Step 9 and 6)

Step 11: {} (Step 10 and 7)

After applying Proof by Refutation i.e., contradicting the conclusion, the problem is terminated with an empty clause ({}). Hence, the conclusion is derived.

Example 3: Show that $\sim S1$ follows from $S1 \rightarrow S2$ and $\sim(S1 \wedge S2)$.

Proof:

Step 1: $S1 \rightarrow S2$ (Premise)

Step 2: $\sim S1 \vee S2$ (Material Implication, Step 1)

Step 3: $\sim(S1 \wedge S2)$ (Premise)

Step 4: $\sim S1 \vee \sim S2$ (De Morgan's, Step 3)

Step 5: $\sim S1$ (Resolution, Step 2, 4)

The resolution mechanism in PL is not used until after the given statements or wffs have been converted into **clausal forms**. To obtain the clausal form of a wff, one must first convert the wff into the Conjunctive Normal Form (CNF). We are already familiar with the fact that a phrase is a formula (and only a formula) of the form: $A_1 \vee A_2 \vee \dots \vee A_n$, where A_i might be either any atomic formula or its negation.

The method of resolution is actually a generalization of Modus Ponens, whose expression is $\frac{P, P \rightarrow Q}{Q}$ which can be written in the equivalent form as

$$\frac{P, \sim P \vee Q}{Q} \text{ (i.e. by using the relation } P \rightarrow Q \Rightarrow \sim P \vee Q \text{).}$$

If we are provided that both P and $\sim P \vee Q$ are true, then we may safely assume that Q is also true. This is a straightforward application of a general resolution principle that will be covered in more detail in this unit.

The construction of a truth table can be used to demonstrate the validity of a resolution process (generally). In order to talk about the resolution process, we will first talk about some of the applications of that method.

Example: Let $C_1: Q \vee R$ and $C_2: \sim Q \vee S$ be two given clauses, so that, one of the literals i.e., Q occurs in one of the clauses (in this case C_1) and its negation ($\sim Q$) occurs in the other clause C_2 . Then application of resolution method in this case tells us to take disjunction of the remaining parts of the given clause C_1 and C_2 , i.e., to take $C_3: R \vee S$ as **deduction** from C_1 and C_2 . Then C_3 is called a **resolvent** of C_1 and C_2 .

The two literals Q and ($\sim Q$) which occur in two different clauses are called **complementary literals**.

In order to illustrate resolution method, we consider another example.

Example: Let us be given the clauses $C_1: \sim S \vee \sim Q \vee R$ and $C_2: \sim P \vee Q$.

In this case, complementary pair of literals viz. Q and $\sim Q$ occur in the two clause C_1 and C_2 .

Hence, the resolution method states: *Conclude* $C_3: \sim S \vee R \vee (\sim P)$

Example: Let us be given the clauses $C_1: \sim Q \vee R$ and $C_2: \sim Q \vee S$

Then, in this case, the clauses do not have any complementary pair of literals and hence, resolution method cannot be applied.

Example: Consider a set of three clauses

$C_1: R$

$C_2: \sim R \vee S$

$C_3: \sim S$

Then, from C_1 and C_2 we conclude, through resolution:

$C_4: S$

From C_3 and C_4 , we conclude,

$C_5: \text{FALSE}$

However, a resolvent FALSE can be deduced only from an unsatisfiable **set of clauses**. Hence, the set of clauses C_1 , C_2 and C_3 is an unsatisfiable set of clauses.

Example: Consider the set of clauses

$C_1: R \vee S$

$C_2: \sim R \vee S$

$C_3: R \vee \sim S$

$C_4: \sim R \vee \sim S$

Then, from clauses C_1 and C_2 we get the resolvent

$C_5: S \vee S = S$

From C_3 and C_4 we get the resolvent

$C_6: \sim S$

From C_5 and C_6 we get the resolvent

$C_7: \text{FALSE}$

Thus, again the set of clauses C_1, C_2, C_3 and C_4 is unsatisfiable.

Note: We could have obtained the resolvent FALSE from only two clauses, viz., C_2 and C_3 . Thus, out of the given four clauses, even set of only two clauses viz., C_2 and C_3 is unsatisfiable. Also, a superset of any unsatisfiable set is unsatisfiable.

Example: Show that the set of clauses:

$C_1: R \vee S$

$C_2: \sim S \vee W$

$C_3: \sim R \vee S$

$C_4: \sim W$ is unsatisfiable.

From clauses C_1 and C_3 we get the resolvent

$C_7: S$

From the clauses C_7 and C_2 we get the resolvent

$C_8: W$

From the clauses C_8 and C_4 we get

$C_9: \text{FALSE}$

Hence, the given set of clauses is unsatisfiable.

Solution of the Problem Using the Resolution Method As was discussed before, the resolution process can also be understood as a refutation approach. The following is an example of a proving technique that can be used to solve problems:

After the symbolic representation of the issue at hand, an additional premise in the form of the negation of the wff, which stands for conclusion, should be added. You can infer either false or a contradiction from this improved set of premises and axioms. If we are able to get to the conclusion that the statement is not true, then the conclusion that was required to be formed is correct, and the issue has been resolved. If, despite our best efforts, we are unable to arrive at the conclusion that the hypothesis is false, then we are unable to determine whether or not the conclusion is correct. As a result, the predicament cannot be solved using the axioms that have been provided and the conclusion that has been drawn.

Let's go on to the next step and apply Resolution Method to the issues we discussed earlier.

Example: If the interest rate goes up, the stock prices might go down. Also, let's say that most people are unhappy when the price of stocks goes down.

Let's say that the rate of interest goes up. Show that most people are unhappy and that we can draw that conclusion.

To show the above conclusion, let us denote the statements as follows:

- A : Interest rate goes up,
- S : Stock prices go down
- U : Most people are unhappy

The problem has the following four statements

- 1) If the interest rate goes up, stock prices go down.
- 2) If stock prices go down, most people are unhappy.
- 3) The interest rate goes up.
- 4) Most people are unhappy. (to conclude)

These statements are first symbolized as wffs of PL as follows:

(1') $A \rightarrow S$

(2') $S \rightarrow U$

(3') A

(4') U (to conclude)

Converting to clausal form, we get

(i) $\sim A \vee S$

(ii) $\sim S \vee U$

(iii) A

(iv) U (to be concluded)

As per resolution method, assume (iv) as false, i.e., assume $\sim U$ as initially given statement, i.e., an axiom.

Thus, the set of axioms in clausal form is:

(i) $\sim A \vee S$

(ii) $\sim S \vee U$

(iii) A

(iv) $\sim U$

Then from (i) and (iii), through resolution, we get the clause
(v) S.

From (ii) and (iv), through resolution, we get the clause

(vi) $\sim S$

From (vi) and (v), through resolution we get,

(viii) FALSE

Hence, the conclusion, i.e., (iv)U: *Most people are unhappy* is valid.

From the above solution using the resolution method, we might have noticed that clausal conversion is a major step that takes a lot of time after translation to wffs. Most of the time, once the clause form is known, proof is easy to see, at least by a person.

☞ Check Your Progress 4

1. Prove that given set of premises are unsatisfiable:

a) $\{X, Y\}, \{\sim X, Z\}, \{\sim X, \sim Z\}, \{X, \sim Y\}$

Ques 2. Consider the given knowledge base to prove the conclusion.

KB1. If Mary goes to school, then Mary eats lunch.

KB 2. If it is Friday, then Mary goes to school or eats lunch.

Conclusion: If it is Friday, then Mary eats lunch.

4.15 ANSWERS/SOLUTIONS

Check Your Progress 1

Ques No	Ans	Ques No	Ans
1	No	8	No
2	No	9	No
3	Yes	10	Yes
4	Yes	11	No
5	No	12	No
6	Yes	13	Yes
7	Yes	14	No

Check Your Progress 2

Ques No	Ans	Ques No	Ans
1	Atomic	6	Atomic
2	Compound	7	Compound
3	Compound	8	Compound
4	Compound	9	Compound
5	Atomic	10	Atomic

Check Your Progress 3

1. $(\sim a_1 \rightarrow (a_1 \vee a_2)) \leftrightarrow a_2$

a_1	a_2	$a_1 \vee a_2$	$\sim a_1 \rightarrow (a_1 \vee a_2)$	$(\sim a_1 \rightarrow (a_1 \vee a_2)) \leftrightarrow a_2$
$_F(0)_$	$_F(0)_$	$_F(0)_$	$_F(0)_$	$_T(1)_$
$_F(0)_$	$_T(1)_$	$_T(1)_$	$_T(1)_$	$_T(1)_$
$_T(1)_$	$_F(0)_$	$_T(1)_$	$_T(1)_$	$_F(0)_$
$_T(1)_$	$_T(1)_$	$_T(1)_$	$_T(1)_$	$_T(1)_$

2. $(\sim \alpha_1 \leftrightarrow (\alpha_2 \leftrightarrow \alpha_3)) \vee (\alpha_3 \wedge \alpha_2)$

α_1	α_2	α_3	$\alpha_2 \leftrightarrow \alpha_3$	$(\sim \alpha_1 \leftrightarrow (\alpha_2 \leftrightarrow \alpha_3))$	$\alpha_3 \wedge \alpha_2$	$(\sim \alpha_1 \leftrightarrow (\alpha_2 \leftrightarrow \alpha_3)) \vee (\alpha_3 \wedge \alpha_2)$
F(0)	F(0)	F(0)	T(1)	T(1)	F(0)	T(1)
T(1)	F(0)	F(0)	T(1)	F(0)	F(0)	F(0)
F(0)	F(0)	T(1)	F(0)	F(0)	F(0)	F(0)
T(1)	F(0)	T(1)	F(0)	T(1)	F(0)	T(1)
F(0)	T(1)	F(0)	F(0)	F(0)	F(0)	F(0)
T(1)	T(1)	F(0)	F(0)	T(1)	F(0)	T(1)
F(0)	T(1)	T(1)	T(1)	T(1)	T(1)	T(1)
T(1)	T(1)	T(1)	T(1)	F(0)	T(1)	T(1)

3. $((\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3) \vee \sim \alpha_4$

α_1	α_2	$\alpha_1 \wedge \alpha_2$	α_3	$((\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3)$	α_4	$((\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3) \vee \sim \alpha_4$
F(0)	F(0)	F(0)	F(0)	T(1)	F(0)	T(1)
F(0)	F(0)	F(0)	F(0)	T(1)	T(1)	T(1)
F(0)	F(0)	F(0)	T(1)	T(1)	F(0)	T(1)
F(0)	F(0)	F(0)	T(1)	T(1)	T(1)	T(1)
F(0)	T(1)	F(0)	F(0)	T(1)	F(0)	T(1)
F(0)	T(1)	F(0)	F(0)	T(1)	T(1)	T(1)
F(0)	T(1)	F(0)	T(1)	T(1)	F(0)	T(1)
F(0)	T(1)	F(0)	T(1)	T(1)	T(1)	T(1)
T(1)	F(0)	F(0)	F(0)	T(1)	F(0)	T(1)
T(1)	F(0)	F(0)	F(0)	T(1)	T(1)	T(1)
T(1)	F(0)	F(0)	T(1)	T(1)	F(0)	T(1)
T(1)	F(0)	F(0)	T(1)	T(1)	T(1)	T(1)
T(1)	T(1)	T(1)	F(0)	F(0)	F(0)	T(1)
T(1)	T(1)	T(1)	F(0)	F(0)	T(1)	F(0)
T(1)	T(1)	T(1)	T(1)	T(1)	F(0)	T(1)
T(1)	T(1)	T(1)	T(1)	T(1)	T(1)	T(1)

4. $((\alpha_1 \rightarrow \sim \alpha_2) \leftrightarrow \alpha_3) \rightarrow \sim (\alpha_1 \vee \alpha_1)$

α_1	α_2	$\alpha_1 \rightarrow \sim \alpha_2$	α_3	$\sim \alpha_1$	$((\alpha_1 \rightarrow \sim \alpha_2) \leftrightarrow \alpha_3)$	$((\alpha_1 \rightarrow \sim \alpha_2) \leftrightarrow \alpha_3) \rightarrow \sim (\alpha_1 \vee \alpha_1)$
F(0)	F(0)	F(0)	F(0)	T(1)	T(1)	T(1)
F(0)	F(0)	F(0)	T(1)	T(1)	F(0)	F(0)
F(0)	T(1)	T(1)	F(0)	T(1)	F(0)	F(0)
F(0)	T(1)	T(1)	T(1)	T(1)	T(1)	T(1)
T(1)	F(0)	T(1)	F(0)	F(0)	F(0)	T(1)
T(1)	F(0)	T(1)	T(1)	F(0)	T(1)	F(0)
T(1)	T(1)	T(1)	F(0)	F(0)	F(0)	F(0)
T(1)	T(1)	T(1)	T(1)	F(0)	T(1)	T(1)

Check Your Progress 4

1. For the given set of premises, derive an empty clause {} using proposition resolution to show that the given set of premises are unsatisfiable.

Step 1: $X \vee Y$ (Premise)
Step 2: $\sim X \vee Z$ (Premise)
Step 3: $\sim X \vee \sim Z$ (Premise)
Step 4: $X \vee \sim Y$ (Premise)
Step 5: X (Resolution on Step 1 and 4)
Step 6: $\sim X$ (Resolution on Step 2 and 3)
Step 7: $\{\}$ (Using Step 5 and 6)

Thus, the given set of premises are unsatisfiable.

2. First introduce notation set for the given knowledge base as:

S: Mary goes to school

L: Mary eats lunch

F: It is Friday

Corresponding knowledge base is:

KB1: $S \rightarrow L$

KB2: $F \rightarrow (S \vee L)$

Conclusion: $F \rightarrow L$

Proof: Step 1: $\sim S \vee L$ (Premise, Material Implication)
Step 2: $\sim F \vee (S \vee L)$ (Premise, Material Implication)
Step 3: F (Negated conclusion)
Step 4: $\sim L$ (Negated conclusion)
Step 5: $(S \vee L)$ (Resolution on Step 2 and 3)
Step 6: L (Resolution on Step 1 and 5)
Step 7: $\{\}$ (Resolution on Step 4 and 6)

4.16 FURTHER READINGS

1. C. L. Liu & D. P. Mohapatra, Elements of Discrete Mathematics: A Computer Oriented Approach, Fourth Edition, 2017, Tata McGraw Hill Education.
2. David J. Hunter, Essentials of Discrete Mathematics, Third Edition, 2016, Jones and Bartlett Publishers.
3. James L. Hein, Discrete Structures, Logic and Computability, Fourth Edition, 2015, Jones and Bartlett Publishers.
4. Kenneth H. Rosen, Discrete Mathematics and Its Applications, Eighth Edition, 2021, Tata McGraw Hill Education.
5. Thomas Koshy, Discrete Mathematics with Applications, 2012, Elsevier Academic Press.

UNIT 5 FIRST ORDER PREDICATE LOGIC

Structure	Page Nos.
5.0	Introduction
5.1	Objectives
5.2	Syntax of First Order Predicate Logic(FOPL)
5.3	Interpretations in FOPL
5.4	Semantics of Quantifiers
5.5	Inference & Entailment in FOPL
5.6	Conversion to clausal form
5.7	Resolution & Unification
5.8	Summary
5.9	Solutions/Answers
5.10	Further/Readings

5.0 INTRODUCTION

In the previous unit, we discussed how propositional logic helps us in solving problems. However, one of the major problems with propositional logic is that, sometimes, it is unable to capture even an elementary type of reasoning or argument as represented by the following statements:

Every man is mortal.

Raman is a man.

Hence, he is mortal.

The above reasoning is intuitively correct. However, if we attempt to simulate the reasoning through Propositional Logic, and further, for this purpose, we use symbols P, Q, and R to denote the statements given above as:

P: Every man is mortal,

Q: Raman is a man,

R: Raman is mortal.

Once the statements in the argument in English are symbolised to apply tools of propositional logic, we have three symbols P, Q, and R, available to us, and apparently no link or connection to the original statements or to each other. The connections, which would have helped in solving the problem, become invisible. In Propositional Logic, there is no way to conclude the *symbol* R from the *symbols* P and Q. However, as we mentioned earlier, even in a natural language, the conclusion of the *statement* denoted by R from *the statements* denoted by P and Q is obvious. Therefore, we search for some **symbolic** system of reasoning that helps us in discussing *argument forms* of the above-mentioned type, in addition to those forms that can be discussed within the framework of propositional logic. **First Order Predicate Logic (FOPL)** is the most well-known symbolic system for this purpose.

The symbolic system of FOPL treats an atomic statement *not as an indivisible unit*. Rather, FOPL not only treats an atomic statement divisible into subject and predicate, but even further deeper structures of an atomic statement are considered in order to handle a larger class of arguments. How and to what extent FOPL symbolizes and establishes *validity/invalidity* and *consistency/inconsistency of arguments* is the subject matter of this unit.

5.1 OBJECTIVES

After studying this unit, you should be able to:

- explain why FOPL is required over and above PL;
- define, and give appropriate examples for, each of the new concepts required for FOPL, including those of quantifier, variable, constant, term, free and bound occurrences of variables, closed and open wff;
- check consistency/validity, if any, of closed formulas;
- reduce a given formula of FOPL to normal forms: Prenex Normal Form (PNF) and (Skolem) Standard Form, and conversion to the clausal form
- use the tools and techniques of FOPL, developed in the unit, to solve problems requiring logical reasoning
- Perform unification and resolution mechanism.

5.2 SYNTAX OF FIRST ORDER PREDICATE LOGIC

We learned about the concept of propositions in Unit 4 of Block 2. Now it's time to understand the difference between the Proposition and the Predicate (also known as a propositional function). In short, a proposition is a specialized statement, whereas a predicate is a generalized statement. To be more specific, the propositions use the logical connectives only, and the predicates use logical connectives and quantifiers (universal and existential), both.

Note: $\exists$ is the symbol used for the Existential quantifier and $\forall$ is used for the Universal quantifier.

Let's understand the difference through some more details, as given below.

A propositional function, or a predicate, in a variable x is a sentence $p(x)$ involving x that becomes a proposition when we give x a definite value from the set of values it can take. We usually denote such functions by $p(x)$, $q(x)$, etc. The set of values x can take is called the universe of discourse.

So, if $p(x)$ is ' $x > 5$ ', then $p(x)$ is not a proposition. But when we give x particular values, say $x = 6$ or $x = 0$, then we get propositions. Here, $p(6)$ is a true proposition and $p(0)$ is a false proposition.

Similarly, if $q(x)$ is ' x has gone to Patna.', then replacing x by 'Taj Mahal' gives us a false proposition.

Note that a predicate is usually not a proposition. But, of course, every proposition is a propositional function in the same way that every real number is a real-valued function, namely, the constant function.

Now, can all sentences be written in symbolic form by using only the logical connectives? What about sentences like ‘x is prime and x + 1 is prime for some x’? How would you symbolize the phrase ‘for some x’, which we can rephrase as ‘there exists an x’? You must have come across this term often while studying mathematics. **We use the symbol ‘ $\exists$ ’ to denote this quantifier, ‘there exists’.** The way we use it is, for instance, to rewrite ‘There is at least one child in the class.’ as ‘ $(\exists x \text{ in } U)p(x)$ ’,

where $p(x)$ is the sentence ‘x is in the class,’ and U is the set of all children.

Now, suppose we take the negative of the proposition we have just stated. Wouldn’t it be ‘There is no child in the class.’? We could symbolize this as ‘for all x in U, $q(x)$ ’ where x ranges over all children and $q(x)$ denotes the sentence ‘x is not in the class.’ i.e., $q(x) \equiv \sim p(x)$.

We have a **mathematical symbol for the quantifier ‘for all’, which is ‘ $\forall$ ’.**

So the proposition above can be written as

‘ $(\forall x \in U)q(x)$ ’, or ‘ $q(x), \forall x \in U$ ’.

An example of the use of the existential quantifier is the true statement.

$(\exists x \in \mathbf{R}) (x + 1 > 0)$, which is read as ‘There exists an x in $\mathbf{R}$ for which $x + 1 > 0$.’

Another example is the false statement

$(\exists x \in \mathbf{N}) (x - \frac{1}{2} = 0)$, which is read as ‘There exists an x in $\mathbf{N}$ for which $x - \frac{1}{2} = 0$.’

An example of the use of the universal quantifier is $(\forall x \notin \mathbf{N}) (x^2 > x)$, which is read as ‘for every x not in $\mathbf{N}$, $x^2 > x$ ’. Of course, this is a false statement, because there is at least one $x \notin \mathbf{N}$, $x \in \mathbf{R}$, for which it is false.

As you have already read in the example of a child in the class,

$(\forall x \in U)p(x)$ is logically equivalent to $\sim (\exists x \in U) (\sim p(x))$. Therefore,

$\sim (\forall x \in U)p(x) \equiv \sim (\exists x \in U) (\sim p(x)) \equiv (\exists x \in U) (\sim p(x))$.

This is one of the rules for negation that relate $\forall$ and $\exists$. The two rules are

$\sim (\forall x \in U)p(x) \equiv (\exists x \in U) (\sim p(x))$, and

$\sim (\exists x \in U)p(x) \equiv (\forall x \in U) (\sim p(x))$

Where U is the set of values that x can take.

5.3 INTERPRETATIONS IN FOPL

In order to have a glimpse at how FOPL extends propositional logic, let us again discuss the earlier argument.

Every man is mortal. Raman is a man.

Hence, he is mortal.

In order to derive the validity of the above simple argument, instead of looking at an atomic statement as indivisible, to begin with, we divide each statement into *subject* and *predicate*. The two predicates that occur in the above argument are:

'is mortal' and *'is man'*.

Let us use the notation

IL: *is_mortal* and

IN: *is_man*.

In view of the notation, the argument on para-phrasing becomes:

For all x, if IN (x) then IL (x).

IN (Raman).

Hence, IL (RAMAN)

More generally, relations of the form *greater-than* (x, y) denoting the phrase ' x is greater than y ', *is_brother_of* (x, y) denoting ' x is brother of y ', *Between* (x, y, z) denoting the phrase that ' x lies between y and z ', and *is_tall* (x) denoting ' x is tall' are some **examples of predicates**. The variables x, y, z etc which appear in a predicate are called **parameters** of the predicate.

The parameters may be given some appropriate values such that after substitution of appropriate values from all possible values of each of the variables, the predicates become *statements*, for each of which we can say whether it is 'True' or it is 'False'.

For example, for the predicate *greater-than* (x, y), if x is given the value 3, then we obtain *greater-than* (3, y), for which it is still not possible to tell whether it is True or False. Hence, '*greater-than* (3, y)' is also a predicate. Further, if the variable y is given the value 5, then we get *greater* (3, 5), which, as we know, is False. Hence, it is possible to give its Truth-value, which is *False* in this case. Thus, from the *predicate greater-than* (x, y), we get the *statement greater-than* (3, 5) by assigning values 3 to the variable x and 5 to the variable y . These values 3 and 5 are called parametric values or *arguments* of the predicate *greater-than*.

(Please note 'argument of a function/predicate' is a mathematical concept, different from logical argument)

Similarly, we can represent the phrase x likes y by the *predicate LIKE* (x, y). Then *Ram likes Mohan* can be represented by the statement *LIKE* (RAM, MOHAN).

Also, *function symbols* can be used in first-order logic. For example, we can use *product* (x, y) to denote $x * y$ and *father* (x) to mean the '*father of x*'. The statement: *Mohan's father loves Mohan* can be symbolised as *LOVE* (*father* (Mohan), Mohan). Thus, we need not know the name of Mohan's father and still we can talk about him. A function serves such a role.

We may note that LIKE (Ram, Mohan) and LOVE (father (Mohan), Mohan) are atoms or atomic statements of PL, in the sense that one can associate a truth-value *True* or *False* with each of these, and each of these does not involve a logical operator like $\sim$, $\wedge$, $\vee$, $\rightarrow$ or $\leftrightarrow$.

Summarizing the above discussion, *LIKE (Ram, Mohan)* and *LOVE (father (Mohan) Mohan)* are **atoms**; whereas GREATER, LOVE, and LIKE are **predicate symbols**; *x* and *y* are **variables** and *3*, *Ram* and *Mohan* are **constants**; and *father* and *product* are **function symbols**.

From the above discussion, we learned the following concepts of symbols.

- i) **Individual symbols or constant symbols**: These are usually names of objects, such as Ram, Mohan, numbers like 3, 5, etc.
- ii) **Variable symbols**: These are usually lowercase, unsubscripted or subscripted letters, like *x*, *y*, *z*, *x*₃.
- iii) **Function symbols**: These are usually lowercase letters like *f*, *g*, *h*,....or strings of lowercase letters such as *father* and *product*.
- iv) **Predicate symbols**: These are usually uppercase letters like *P*, *Q*, *R*,....or strings of lowercase letters such as *greater-than*, *is_tall* etc.

A function symbol or predicate symbol takes a fixed number of arguments. If a function symbol *f* takes *n* arguments, *f* is called an *n-place function symbol*. Similarly, if a predicate symbol *Q* takes *m* arguments, *P* is called an *m-place predicate symbol*. For example, *father* is a one-place function symbol, and GREATER and LIKE are two-place predicate symbols. However, *father-of* in *father_of (x, y)* is a two place predicate symbol.

The symbolic representation of an argument of a function or a predicate is called a *term*, where a **term** is defined recursively as follows:

- i) A variable is a term.
- ii) A constant is a term.
- iii) If *f* is an *n*-place function symbol, and *t*₁....*t*_{*n*} are terms, then *f(t*₁,....*t*_{*n*}) is a term.
- iv) Any term can be generated only by the application of the rules given above.

For example, since *y* and *3* are both terms and *plus* is a two-place function symbol, *plus (y, 3)* is a term according to the above definition.

Furthermore, we can see that *plus (plus (y, 3), y)* and *father (father (Mohan))* are also terms; the former denotes (*y* + 3) + *y* and the latter denotes *grandfather of Mohan*.

A predicate can be thought of as a function that maps a list of constant arguments to T or F. For example, GREATER is a predicate with GREATER (5, 2) as T, but GREATER (1, 3) as F.

We already know that in PL, an atom or atomic statement is an indivisible unit for representing and validating arguments. Atoms in PL are generally denoted by symbols like *P*, *Q*, and *R*, etc. But in FOPL,

Definition: An Atom is

- (i) either an atom of Propositional Logic, or
- (ii) is obtained from an n -place predicate symbol P , and terms $t_1, \dots, t_n$ so that $P(t_1, \dots, t_n)$ is an atom.

Once the atoms are defined, we can build complex formulas of FOPL by using the logical connectives defined in Propositional Logic and assuming that they have similar meanings in FOPL. Two special symbols $\forall$ and $\exists$ are used to denote quantifications in FOPL. Two special symbol $\forall$ and $\exists$ are used to denote qualifications in FOPL. The symbols $\forall$ and $\exists$ are called, respectively, the *universal* quantifier and *existential* quantifier. For a variable x , $(\forall x)$ is read as *for all x* , and $(\exists x)$ is read as *there exists an x* . Next, we consider some examples to illustrate the concepts discussed above.

In order to symbolize the following statements:

- i) There exists a number that is rational.
- ii) Every rational number is a real number
- iii) For every number x , there exists a number y , which is greater than x .

Let us denote *x is a rational number* by $Q(x)$, *x is a real number* by $R(x)$, and *x is less than y* by $LESS(x, y)$. Then the above statements may be symbolized respectively, as

- (i) $(\exists x) Q(x)$
- (ii) $(\forall x) (Q(x) \rightarrow R(x))$
- (iii) $(\forall x) (\exists y) LESS(x, y)$.

Each of the expressions (i), (ii), and (iii) is called a **formula** or a well-formed formula or **wff**.

5.4 SEMANTICS OF QUANTIFIERS

To understand the semantics of quantifiers, we need to first understand the difference between a proposition and a predicate (also known as a propositional function). In short, a proposition is a specialized statement, whereas a predicate is a generalized statement. To be more specific, the propositions use only logical connectives, whereas predicates use both logical connectives and quantifiers (universal and existential).

Note: $\exists$ is the symbol used for the Existential quantifier and $\forall$ is used for the Universal quantifier.

Let's understand the difference through some more details, as given below.

A propositional function, or a **predicate**, in a variable x is a sentence $p(x)$ involving x that becomes a proposition when we give x a definite value from the set of values it can take. We usually denote such functions by $p(x)$, $q(x)$, etc. The set of values x can take is called the universe of discourse.

So, if $p(x)$ is ' $x > 5$ ', then $p(x)$ is not a proposition. But when we give x particular values, say $x = 6$ or $x = 0$, then we get propositions. Here, $p(6)$ is a true proposition and $p(0)$ is a false proposition.

Similarly, if $q(x)$ is 'x has gone to Patna.', then replacing x by 'Taj Mahal' gives us a false proposition.

Note that a predicate is usually not a proposition. But, of course, every proposition is a propositional function in the same way that every real number is a real-valued function, namely, the constant function.

Now, can all sentences be written in symbolic form by using only the logical connectives? What about sentences like 'x is prime and $x + 1$ is prime for some x'? How would you symbolize the phrase 'for some x', which we can rephrase as 'there exists an x'? You must have come across this term often while studying mathematics. **We use the symbol ' $\exists$ ' to denote this quantifier, 'there exists'.** The way we use it is, for instance, to rewrite 'There is at least one child in the class.' as ' $(\exists x \text{ in } U)p(x)$ ',

where $p(x)$ is the sentence 'x is in the class,' and U is the set of all children.

Now, suppose we take the negative of the proposition we have just stated. Wouldn't it be 'There is no child in the class.'? We could symbolize this as 'for all x in U, $q(x)$ ' where x ranges over all children and $q(x)$ denotes the sentence 'x is not in the class.', i.e., $q(x) \equiv \sim p(x)$.

We have a **mathematical symbol for the quantifier 'for all', which is ' $\forall$ '.**

So the proposition above can be written as

' $(\forall x \in U)q(x)$ ', or ' $q(x), \forall x \in U$ '.

An example of the use of the existential quantifier is the true statement.

$(\exists x \in \mathbf{R}) (x + 1 > 0)$, which is read as 'There exists an x in $\mathbf{R}$ for which $x + 1 > 0$.'

Another example is the false statement

$(\exists x \in \mathbf{N}) (x - \frac{1}{2} = 0)$, which is read as 'There exists an x in $\mathbf{N}$ for which $x - \frac{1}{2} = 0$.'

An example of the use of the universal quantifier is $(\forall x \notin \mathbf{N}) (x^2 > x)$, which is read as 'for every x not in $\mathbf{N}$, $x^2 > x$.' Of course, this is a false statement, because there is at least one $x \notin \mathbf{N}$, $x \in \mathbf{R}$, for which it is false.

As you have already read in the example of a child in the class,

$(\forall x \in U)p(x)$ is logically equivalent to $\sim (\exists x \in U) (\sim p(x))$. Therefore,

$\sim (\forall x \in U)p(x) \equiv \sim \sim (\exists x \in U) (\sim p(x)) \equiv (\exists x \in U) (\sim p(x))$.

This is one of the rules for negation that relate $\forall$ and $\exists$. The two rules are

$\sim (\forall x \in U)p(x) \equiv (\exists x \in U) (\sim p(x))$, and

$\sim (\exists x \in U)p(x) \equiv (\forall x \in U) (\sim p(x))$

Where U is the set of values that x can take.

Next, we discuss three new concepts, viz **Scope** of occurrence of a quantified variable, **Bound** occurrence of a quantifier variable or quantifier and **Free occurrence** of a variable.

Before discussing these concepts, we should know the *difference between a variable and the occurrence of a variable in a quantifier expression*.

The variable x has THREE occurrences in the formula

$$(\exists x) Q(x) \rightarrow P(x, y).$$

Also, the variable y has only one occurrence, and the variable z has zero occurrences in the above formula. Next, we define the three concepts mentioned above.

Scope of an occurrence of a quantifier is the smallest but complete formula following the quantifier, sometimes delimited by a pair of parentheses. For example, $Q(x)$ is the scope of $(\exists x)$ in the formula

$$(\exists x) Q(x) \rightarrow P(x, y).$$

But the scope of $(\exists x)$ in the formula: $(\exists x) (Q(x) \rightarrow P(x, y))$ is $(Q(x) \rightarrow P(x, y))$.

Further in the formula:

$$(\exists x) (P(x) \rightarrow Q(x, y)) \wedge (\exists x) (P(x) \rightarrow R(x, 3)),$$

the scope of **first** occurrence of $(\exists x)$ is the formula $(P(x) \rightarrow Q(x, y))$ and the scope of **second** occurrence of $(\exists x)$ is the formula

$$(P(x) \rightarrow R(x, 3)).$$

As another example, the scope of the only occurrence of the quantifier $(\forall y)$ in $(\exists x) ((P(x) \rightarrow Q(x) \leftrightarrow (\forall y) (Q(x) \rightarrow R(y))))$ is $(Q(x) \rightarrow R(y))$. But the scope of the only occurrence of the existential variable $(\exists x)$ in the same formula is the formula:

$$(P(x) \rightarrow Q(x)) \wedge (\forall y) (Q(x) \rightarrow R(y))$$

An **occurrence** of a variable in a formula is **bound** if and only if the occurrence is within the scope of a quantifier employing the variable, or is the occurrence in that quantifier. An occurrence of a variable in a formula is **free** if and only if this occurrence of the variable is not bound.

Thus, in the formula $(\exists x) P(x, y) \rightarrow Q(x)$, there are three occurrences of x , out of which the first two occurrences of x are *bound*, whereas the last occurrence of x is *free*, because the scope of $(\exists x)$ in the above formula is $P(x, y)$. The only occurrence of y in the formula is free. Thus, x is both a bound and a free variable in the above formula, and y is only a free variable in the formula so far. We talked of an *occurrence of a variable* as free or bound. Now, we talk of (only) a *variable* as free or bound. A variable is **free** in a formula if at least one of its occurrences is free in the formula. A variable is **bound** in a formula if at least one of its occurrences is bound.

It may be noted that a variable can be **both free and bound** in a formula. In order to further elucidate the concepts of *scope*, *free and bound occurrences of a variable*, we consider a similar but different formula for this purpose:

$$(\exists x) (P(x, y) \rightarrow Q(x)).$$

In this formula, the *scope* of the only occurrence of the quantifier $(\exists x)$ is the whole of the rest of the formula, viz., the scope of $(\exists x)$ in the given formula is $(P(x, y) \rightarrow Q(x))$

Also, all three occurrences of the variable x are bound. The only occurrence of y is free.

Remarks: It may be noted that a bound variable x is just a **placeholder** or a **dummy variable** in the sense that all occurrences of a bound variable x may be replaced by another free variable, say y , which does not occur in the formula. However, once x is replaced by y , then y becomes bound. For example, $(\forall x) (f(x))$ is the same as $(\forall y) f(y)$. It is something like

$$\int_1^2 x^2 dx = \int_1^2 y^2 dy = \frac{2^3}{3} - \frac{1^3}{3} = \frac{7}{3}$$

Replacing a bound variable x by another variable y under the restrictions mentioned above is called **the renaming of a variable x** .

Having defined an atomic formula of FOPL, next, we consider the definition of a general formula formally in terms of atoms, logical connectives, and quantifiers.

Definition: A well-formed formula (**wff**) in FOPL is defined recursively as follows:

- i) An atom or atomic formula is a *wff*.
- ii) If E and G are *wff*, then each of $\sim (E)$, $(E \vee G)$, $(E \wedge G)$, $(E \rightarrow G)$, $(E \leftrightarrow G)$ is a **wff**.
- iii) If E is a *wff* and x is a free variable in E , then $(\forall x)E$ is a *wff*.
- iv) A *wff* can be obtained only by applications of (i), (ii), and (iii) given above.

We may drop pairs of parentheses by agreeing that quantifiers have the least possible scope. For example, $(\exists x) P(x, y) \rightarrow Q(x)$ stands for $((\exists x) P(x, y)) \rightarrow Q(x)$

We may note the following two cases of translation:

- (i) for all x , $P(x)$ is $Q(x)$ is translated as $(\forall x) (P(x) \rightarrow Q(x))$
(the other possibility $(\forall x) P(x) \wedge Q(x)$ is not valid.)
- (ii) for some x , $P(x)$ is $Q(x)$ is translated as $(\exists x) P(x) \wedge Q(x)$
(the other possibility $(\exists x) P(x) \rightarrow Q(x)$ is not valid)

Example:

Translate the statement: *Every man is mortal. Raman is a man. Therefore, Raman is mortal.*

As discussed earlier, let us denote “*x is a man*” by $MAN(x)$, and “*x is mortal*” by $MORTAL(x)$. Then “*every man is mortal*” can be represented by

$$(\forall x) (MAN(x) \rightarrow MORTAL(x)),$$

“Raman is a man” by

$$MAN(Raman).$$

The whole argument can now be represented as:

$$(\forall x) (MAN(x) \rightarrow MORTAL(x)) \wedge MAN(Raman) \rightarrow MORTAL(Raman).$$

as a single statement.

In order to further explain symbolisation, let us recall the axioms of natural numbers:

- (1) For every number, there is one and only one immediate successor,
- (2) There is no number for which 0 is the immediate successor.
- (3) For every number other than 0, there is one and only one immediate predecessor.

Let the *immediate successor* and *predecessor* of x , respectively, be denoted by $f(x)$ and $g(x)$.

Let $E(x, y)$ denote x is *equal to* y . Then the axioms of natural numbers are represented respectively by the formulas:

- (i) $(\forall x) (\exists y) (E(y, f(x)) \wedge (\forall z) (E(z, f(x)) \rightarrow E(y, z)))$
- (ii) $\sim ((\exists x) E(0, f(x)))$ and
- (iii) $(\forall x) (\sim E(x, 0) \rightarrow ((\exists y) E(y, g(x)) \wedge (\forall z) (E(z, g(x)) \rightarrow E(y, z))))$.

From the semantics (for meaning or interpretation) point of view, the **wff of FOPL** may be divided into two categories, each consisting of

- (i) wffs, in each of which, **all** occurrences of variables are **bound**.
- (ii) wffs, in each of which at **least one** occurrence of a variable is **free**.

The wffs of FOPL in which there is no occurrence of a free variable are like wffs of PL in the sense that we can call each wff **True, False, consistent, inconsistent, valid, invalid, etc.** Each such formula is called a **closed formula**. However, when a wff involves a free occurrence, then it is not possible to call such a wff as True, False, etc. **Each of such a formula is called an open formula.**

For example, each of the formulas greater (x, y), greater (x, 3), (y) greater (x, y) has one free occurrence of the variable x. Hence, each is an open formula.

Each of the formulas: $(\forall x) (\exists y) \text{ greater } (x, y)$, $(\forall y) \text{ greater } (y, 1)$, $\text{greater } (9, 2)$, does not have free occurrence of any variable. Therefore each of these formulas is a closed formula.

Next, we discuss some equivalences and inequalities

The following equivalences hold for any two formulas $P(x)$ and $Q(x)$:

$$(i) (\forall x) P(x) \wedge (\forall x) Q(x) = (\forall x) (P(x) \wedge Q(x))$$

$$(ii) (\exists x) P(x) \vee (\exists x) Q(x) = (\exists x) (P(x) \vee Q(x))$$

But the following inequalities hold, in general:

$$(iii) (\forall x) (P(x) \vee Q(x)) \neq (\forall x) P(x) \vee (\forall x) Q(x)$$

$$(iv) (\exists x) (P(x) \wedge Q(x)) \neq (\exists x) P(x) \wedge (\exists x) Q(x)$$

We justify (iii) & (iv) below:

Let $P(x)$: x is odd natural number,

$Q(x)$: x is an even natural number.

Then L.H.S of (iii) above states *for every natural number it is either odd or even, which is correct*. But R.H.S of (iii) states that *every natural number is odd or every natural number is even, which is not correct*.

Next, L.H.S. of (iv) states that there is a natural number which is both even and odd, **which is not correct**. However, the R.H.S. of (iv) says that there is an integer that is odd and an integer that is even, which is correct.

Equivalences involving Negation of Quantifiers

$$(v) \sim (\forall x) P(x) = (\exists x) \sim P(x)$$

$$(vi) \sim (\exists x) P(x) = (\forall x) \sim P(x)$$

Examples: For each of the following closed formulas, prove

(i) $(\forall x) P(x) \wedge (\exists y) \sim P(y)$ is inconsistent.

(ii) $(\forall x) P(x) \rightarrow (\exists y) P(y)$ is valid

Solution: (i) Consider

$$\begin{aligned} & (\forall x) P(x) \wedge (\exists y) \sim P(y) \\ &= (\forall x) P(x) \wedge \sim (\forall y) P(y) \quad (\text{taking negation out}) \end{aligned}$$

But we know that for each bound occurrence, a variable is dummy variable and can be replaced in the whole scope of the variable uniformly by another free variable. Hence,

$$R = (\forall x) P(x) \wedge \sim (\forall x) P(x)$$

Each conjunct of the formula is either True or False and, hence, can be thought of as a formula of PL, instead of a formula of FOPL. Let us replace $(\forall x) P(x)$ by Q , a formula of PL.

$$R = Q \wedge \sim Q = \text{False}$$

Hence, the result is proved.

(ii) Consider

$(\forall x) P(x) \rightarrow (\exists y) P(y)$
 Replacing ' $\rightarrow$ ' we get
 $= \sim (\forall x) P(x) \vee (\exists y) P(y)$
 $= (\exists x) \sim P(x) \vee (\exists y) P(y)$
 $= (\exists x) \sim P(x) \vee (\exists x) P(x)$ (renaming x as y in the second disjunct)

In other words,

$= (\exists x) (\sim P(x) \vee P(x))$ (using equivalence)

The last formula states: *there is at least one element, say b , for which $\sim P(b) \vee P(b)$ holds, i.e., for b , either $P(b)$ is False or $P(b)$ is True.*

But, as P is a predicate symbol and b is a constant, $\sim P(b) \vee P(b)$ must be True. Hence, the proof.

Check Your Progress - 1

Ex. 1 Let $P(x)$ and $Q(x)$ represent “ x is a rational number” and “ x is a real number,” respectively. Symbolize the following sentences:

- (i) Every rational number is a real number.
- (ii) Some real numbers are rational numbers.
- (iii) Not every real number is a rational number.

Ex. 2 Let $C(x)$ mean “ x is a used-car dealer,” and $H(x)$ mean “ x is honest.” Translate each of the following into English:

- (i) $(\exists x)C(x)$
- (ii) $(\exists x) H(x)$
- (iii) $(\forall x)C(x) \rightarrow \sim H(x)$
- (iv) $(\exists x) (C(x) \wedge H(x))$
- (v) $(\exists x) (H(x) \rightarrow C(x))$.

Ex. 3 Prove the following:

- (i) $P(a) \rightarrow \sim ((\exists x) P(x))$ is consistent.
- (ii) $(\forall x) P(x) \vee ((\exists y) \sim P(y))$ is valid.

5.5 INFERENCE & ENTAILMENT IN FOPL

In the previous unit, we discussed eight inferencing rules of Propositional Logic (PL) and further discussed applications of these rules in exhibiting the validity/invalidity of arguments in **PL**. In this section, the earlier eight rules are extended to include four more rules involving quantifiers for inferencing. Each

of the new rules is called a **Quantifier Rule**. The extended set of 12 rules is then used for validating arguments in First Order Predicate Logic (**FOPL**).

Before introducing and discussing the Quantifier rules, we briefly discuss why, at all, these rules are required. For this purpose, let us recall the argument discussed earlier, which Propositional Logic could not handle:

- (i) Every man is mortal.
- (ii) Raman is a man.
- (iii) Raman is mortal.

The equivalent symbolic form of the argument is given by:

$$(i') (\forall x) (\text{Man}(x) \rightarrow \text{Mortal}(x))$$

$$(ii') \text{Man}(\text{Raman})$$

$$(iii') \text{Mortal}(\text{Raman})$$

If, instead of (i') we were given

$$(iv) \text{Man}(\text{Raman}) \rightarrow \text{Mortal}(\text{Raman}),$$

(which is a formula of Propositional Logic, also), then using Modus Ponens on (ii') & (iv) in Propositional Logic, we would have obtained (iii') Mortal (Raman).

However, from (i') & (ii') we cannot derive in Propositional Logic (iii'). This suggests that there should be mechanisms for dropping and introducing quantifier appropriately, i.e., in such a manner that *validity* of arguments is not violated. Without discussing the validity-preserving characteristics, we introduce the four Quantifier rules.

(i) Universal Instantiation Rule (U.I.):

$$\frac{(\forall x)p(x)}{p(a)}$$

Where a is an arbitrary constant.

The rule states that if $(\forall x)p(x)$ is True, then we can assume $P(a)$ as True for any constant a (where a constant a is like Raman). It can be easily seen that the rule associates a formula $P(a)$ of Propositional Logic to a formula $(\forall x)p(x)$ of FOPL. The significance of the rule lies in the fact that once we obtain a formula like $P(a)$, then the reasoning process of Propositional Logic may be used. The rule may be used whenever its application seems to be appropriate.

(ii) Universal Generalisation Rule (U.G.)

$$\frac{P(a), \text{ for all } a}{(\forall x)p(x)}$$

The rule says that if it is known that for all constants a , the statement $P(a)$ is True, then we can, instead, use the formula $(\forall x)p(x)$.

The rule is associated with a set of formulas $P(a)$ for all a of Propositional Logic, a formula $(\forall x)p(x)$ of FOPL.

Before using the rule, we must ensure that $P(a)$ is True for all a , otherwise, it may lead to wrong conclusions.

(iii) Existential Instantiation Rule (E. I.)

$$\frac{(\exists x) P(x)}{P(a)} \quad (E.I.)$$

The rule says if the Truth of $(\exists x) P(x)$ is known, then we can assume the Truth of $P(a)$ for **some fixed** a . The rule, again, associates a formula $P(a)$ of Propositional Logic to a formula $(\exists x)p(x)$ of FOPL.

An inappropriate application of this rule may lead to *wrong* conclusions. The source of possible errors lies in the fact that the choice 'a' in the rule is *not arbitrary* and can not be known at the time of deducing $P(a)$ from $(\exists x) P(x)$.

If during the process of deduction, some other $(\exists y) Q(y)$ or $(\exists x) (R(x))$ or even another $(\exists x)P(x)$ is encountered, then each time a new constant, say b , c , etc., should be chosen to infer $Q(b)$ from $(\exists y) Q(y)$ or $R(c)$ from $(\exists x) (R(x))$ or $P(d)$ from $(\exists x) P(x)$.

(iv) Existential Generalization Rule (E.G)

$$\frac{P(a)}{(\exists x)P(x)} \quad (E.G)$$

The rule states that if $P(a)$, a formula of Propositional Logic is True, then the Truth of $(\exists x) P(x)$, a formula of FOPL, may be assumed to be True.

The Universal Generalisation (U.G) and Existential Instantiation (E.I.) rules should be applied with utmost care; however, the other two rules may be applied whenever it appears to be appropriate.

Next, the purpose of the two rules, viz.,

(i) Universal Instantiation Rule (U. I.)

(iii) Existential Rule (E. I.)

is to associate formulas of Propositional Logic (PL) to formulas of FOPL in a manner that the validity of arguments due to these associations is not disturbed. Once we get formulas of PL, then any of the eight rules of inference of PL may be used to validate conclusions and solve problems requiring logical reasoning for their solutions.

The purpose of the other quantification rules, viz., for generalisation, i.e.,

$$(ii) \frac{P(a), \text{ for all } a}{(\forall x) P(x)}$$

$$(iv) \frac{P(a)}{(\exists x) P(x)}$$

is to derive conclusions that are formulas of FOPL rather than formulas of PL. However, while making inference, we may first associate formulas of PL with formulas of FOPL and then use inference rules of PL to conclude formulas in PL. But the conclusion to be made in the problem may correspond to a formula of FOPL. These two generalisation rules help us in associating formulas of FOPL with formulas of PL.

Example: State, with reasons, which of the following are correct inferences and which are not.

- (i) To conclude $F(a) \wedge G(a) \rightarrow H(a) \wedge I(a)$
from $(\forall x)(F(x) \wedge G(x)) \rightarrow H(x) \wedge I(x)$

using Universal Instantiation (U.I.) rule

The above inference or conclusion is *incorrect* in view of the fact that the scope of universal quantification is only the formula: $F(x) \wedge G(x)$ and not the whole of the formula.

The occurrences of x in $H(x) \wedge I(x)$ are free occurrences. Thus, one of the correct inferences would have been:

$$F(a) \wedge G(a) \rightarrow H(x) \wedge I(x)$$

- (ii) To conclude $F(a) \wedge G(a) \rightarrow H(a) \wedge I(a)$ from
 $(\forall x)(F(x) \wedge G(x) \rightarrow H(x) \wedge I(x))$ using U.I.

The conclusion is correct in view of the argument given in (i) above.

- (iii) To conclude $\sim F(a)$ for an arbitrary a , from $\sim(\forall x)F(x)$ using U.I.

The conclusion is incorrect, because actually

$$\sim(\forall x)F(x) = (\exists x)\sim F(x)$$

Thus, the inference is not a case of U.I., but of Existential Instantiation (E.I.)

Further, as per restrictions, we can not say for which a , $\sim F(x)$ is True. Of course, $\sim F(x)$ is true for some constant, but not necessarily for a pre-assigned constant a .

- (iv) to conclude $((F(b) \wedge G(b)) \rightarrow H(c))$
from $(\exists x)((F(b) \wedge G(x)) \rightarrow H(c))$

using E.I. is *not* correct

The reason is that the constant to be substituted for x cannot be assumed to be the same constant b , being given in advance, as an argument of F . However,

to conclude $((F(b) \wedge G(a)) \rightarrow H(c))$

from $(\exists x)((F(b) \wedge G(x)) \rightarrow H(c))$ is correct.

Step for using Predicate Calculus as a Language for Representing Knowledge & for Reasoning:

Step 1: Conceptualisation: First of all, all the relevant entities and the relations that exist between these entities are explicitly enumerated. Some of the implicit facts like, 'a person dead once is dead forever' have to be explicated.

Step 2: Nomenclature & Translation: Giving appropriate names to the objects and relations, and then translating the given sentences in English to formulas in FOPL. Appropriate names are essential in order to guide a reasoning system based on FOPL. It is well-established that no reasoning system is complete. In other words, a reasoning system may need help in arriving at the desired conclusion.

Step 3: Finding an appropriate sequence of reasoning steps, involving the selection of an appropriate rule and appropriate FOPL formulas to which the selected rule is to be applied, to reach the conclusion.

Applications of the 12 inference rules (8 of Propositional Logic and 4 involving Quantifiers)

Example: Symbolize the following and then construct a proof for the argument:

- (i) Anyone who repairs their own car is highly skilled and saves a lot of money on repairs
- (ii) Some people who repair their own cars have menial jobs. Therefore,
- (iii) Some people with menial jobs are highly skilled.

Solution: Let us use the notation:

- P(x) : x is a person
- S(x) : x saves money on repairs
- M(x) : x has a menial job
- R(x) : x repairs his own car
- H(x) : x is highly skilled.

Therefore, (i), (ii) and (iii) can be symbolized as:

- (i) $(\forall x) (R(x) \rightarrow (H(x) \wedge S(x)))$
- (ii) $\exists(x) (R(x) \wedge M(x))$
- (iii) $(\exists x) (M(x) \wedge H(x))$ (to be concluded)

From (ii) using Existential Instantiation (E.I), we get, for some fixed a

- (iv) $R(a) \wedge M(a)$

Then, by simplification rule of Propositional Logic, we get

(v) $R(a)$

From (i), using *Universal Instantiation (U.I.)*, we get

(vi) $R(a) \rightarrow H(a) \wedge S(a)$

Using *modus ponens w.r.t. (v) and (vi)* we get

(vii) $H(a) \wedge S(a)$

By *specialisation of (vii)* we get

(viii) $H(a)$

By *specialisation of (iv)* we get

(ix) $M(a)$

By *conjunctions of (viii) & (ix)* we get

$M(a) \wedge H(a)$

By *Existential Generalisation*, we get

$(\exists x) (M(x) \wedge H(x))$

Hence, (iii) is concluded.

Example:

- (i) Some juveniles who commit minor offences are thrown into prison, and any juvenile thrown into prison is exposed to all sorts of hardened criminals.
- (ii) A juvenile who is exposed to all sorts of hardened criminals will become bitter and learn more techniques for committing crimes.
- (iii) Any individual who learns more techniques for committing crimes is a menace to society, if he is bitter.
- (iv) Therefore, some juveniles who commit minor offences will be menaces to the society.

Example: Let us symbolize the statement in the given argument as follows:

- (i) $J(x)$: x is juvenile.
- (ii) $C(x)$: x commits minor offences.
- (iii) $P(x)$: x is thrown into prison.
- (iv) $E(x)$: x is exposed to hardened criminals.
- (v) $B(x)$: x becomes bitter.
- (vi) $T(x)$: x learns more techniques for committing crimes.
- (vii) $M(x)$: x is a menace to society.

The statements of the argument may be translated as:

- (i) $(\exists x) (J(x) \wedge C(x) \wedge P(x)) \wedge ((\forall y) (J(y) \rightarrow E(y)))$
 - (ii) $(\forall x) (J(x) \wedge E(x) \rightarrow B(x) \wedge T(x))$
 - (iii) $(\forall x) (T(x) \wedge B(x) \rightarrow M(x))$
- Therefore,

$$(iv) \quad (\exists x) (J(x) \wedge C(x) \wedge M(x))$$

By simplification (i) becomes

$$(v) \quad (\exists x) (J(x) \wedge C(x) \wedge P(x)) \text{ and}$$

$$(vi) \quad (\forall y) (J(y) \rightarrow E(y))$$

From (v) through Existential Instantiation, for some fixed b , we get

$$(vii) \quad J(b) \wedge C(b) \wedge P(b)$$

Through simplification (vii) becomes

$$(viii) \quad J(b)$$

$$(ix) \quad C(b) \text{ and}$$

$$(x) \quad P(b)$$

Using Universal Instantiation, on (vi), we get

$$(xi) \quad J(b) \rightarrow E(b)$$

Using Modus Ponens in (vii) and (xi) we get

$$(xii) \quad E(b)$$

Using conjunction for (viii) & (xii) we get

$$(xiii) \quad J(b) \wedge E(b)$$

Using Universal Instantiation on (ii) we get

$$(xiv) \quad J(b) \wedge E(b) \rightarrow B(b) \wedge T(b)$$

Using Modus Ponens for (xiii) & (xiv), we get

$$(xv) \quad T(b) \wedge B(b)$$

Using Universal Instantiation for (iii) we get

$$(xvi) \quad T(b) \wedge B(b) \rightarrow M(b)$$

Using Modus Ponens with (xv) and (xvi) we get

$$(xvii) \quad M(b)$$

Using conjunction for (viii), (ix) and (xvii) we get

$$(xviii) \quad J(b) \wedge C(b) \wedge M(b)$$

From (xviii), through Existential Generalization we get the required (iv), i.e.

$$(\exists x) (J(x) \wedge C(x) \wedge M(x))$$

Remark: It may be noted the occurrence of quantifiers is not, in general, commutative i.e.,

$$(Q_1x) (Q_2x) \neq (Q_2x) (Q_1x)$$

For example

$$(\forall x) (\exists y) F(x,y) \neq (\exists y) (\forall x) F(x,y) \quad (A)$$

The occurrence of $(\exists y)$ on L.H.S depends on x i.e., occurrence of y on L.H.S is a function of x . However, the occurrence of $(\exists y)$ on R.H.S is independent of x , hence, occurrence of y on R.H.S is not a function of x .

For example, if we take $F(x,y)$ to mean:

y and x are integers such that $y > x$,

then, *L.H.S of (A) above states: For each x there is a y such that $y > x$.*

The statement is true in the domain of real numbers.

On the other hand, *R.H.S of (A) above states that: There is an integer y which is greater than x, for all x.*

This statement is not true in the domain of real numbers.

When the logical statements are interconnected in a manner that one is a consequence of other then such Logical consequences (also called entailment) are the fundamental concept in logical reasoning, which describes the relationship between statements that hold true when one statement logically follows from one or more statements.

A valid logical argument is one in which the conclusion is entailed by the premises, because the conclusion is the consequence of the premises. The philosophical analysis of logical consequence involves questions such as: In what sense does a conclusion follow from its premises? and What does it mean for a conclusion to be a consequence of premises? All of philosophical logic is concerned with providing accounts of the nature of logical consequence and the nature of logical truth

Logical consequence is necessary and formal, as illustrated by examples explained through formal proofs and models of interpretation. A sentence is said to be a logical consequence of a set of sentences, for a given language, if and only if, using only logic (i.e., without regard to any personal interpretations of the sentences) the sentence must be true if every sentence in the set is true.

5.6 CONVERSION TO CLAUSAL FORM

In order to facilitate problem solving through Propositional Logic, we discussed two normal forms, viz, the conjunctive normal form **CNF** and the disjunctive normal form **DNF**. In **FOPL**, there is a normal form called the **prenex normal form**. Further the statement in Prenex Normal Form is required to be skolemized to get the clausal form, which can be used for the purpose of Resolution.

So, the first step towards the clausal form is to begin with Prenex Normal Form (PNF), and the second step is Skolemization, which will be discussed after PNF.

Prenex Normal Form (PNF): In a broad sense, it relates to re-alignment of the quantifiers, i.e. to bring all the quantifiers in the beginning of the expression and then replacing the existential and universal quantifiers with constants and the functions is performed for Skolemization i.e., to bring the statement in clausal form.

The use of a prenex normal form of a formula simplifies the proof procedures, to be discussed.

Definition A formula G in FOPL is said to be in a **prenex normal form** if and only if the formula G is in the form

$$(Q_1x_1) \dots (Q_nx_n) P$$

where each (Q_ix_i) , for $i = 1, \dots, n$, is either $(\forall x_i)$ or $(\exists x_i)$, and P is a quantifier free formula. The expression $(Q_1x_1) \dots (Q_nx_n)$ is called the **prefix** and P is called the **matrix of the formula G**.

Examples of some formulas in prenex normal form are as follows:

- (i) $(\exists x) (\forall y) (R(x, y) \vee Q(y)), (\forall x) (\forall y) (\sim P(x, y) \rightarrow S(y)),$
- (ii) $(\forall x) (\forall y) (\exists z) (P(x, y) \rightarrow R(z)).$

Next, we consider a method of transforming a given formula into a prenex normal form. For this, first we discuss equivalence of formulas in FOPL. Let us recall that two formulas E and G are **equivalent**, denoted by $E = G$, *if and only if the truth values of E and G are identical under every interpretation*. The pairs of equivalent formulas given in Table of equivalent Formulas of previous unit are still valid as these are quantifier-free formulas of FOPL. However, there are pairs of equivalent formulas of FOPL that contain quantifiers. Next, we discuss these additional pairs of equivalent formulas. We introduce some notation specific to FOPL: *the symbol G denotes a formula that does not contain any free variable x* . Then we have the following pairs of equivalent formulas, where Q denotes a quantifier which is either $\forall$ or $\exists$. Next, we introduce four laws for **pairs of equivalent formulas**.

In the rest of the discussion of FOPL, $P[x]$ is used to denote the fact that x is a free variable in the formula P , for example, $P[x] = (\forall y) P(x, y)$. Similarly, $R[x, y]$ denotes that variables x and y occur as free variables **in the formula R** . Some of these equivalences, we have discussed earlier.

Then, the following laws involving quantifiers hold good in FOPL

- (i) $(Qx) P[x] \vee G = (Qx) (P[x] \vee G).$
- (ii) $(Qx) P[x] \wedge G = (Qx) (P[x] \wedge G).$

In the above two formulas, Q may be either $\forall$ or $\exists$.

- (iii) $\sim ((\forall x) P[x]) = (\exists x) (\sim P[x]).$
- (iv) $\sim ((\exists x) P[x]) = (\forall x) (\sim P[x]).$
- (v) $(\forall x) P[x] \wedge (\forall x) H[x] = (\forall x) (P[x] \wedge H[x]).$
- (vi) $(\exists x) P[x] \vee (\exists x) H[x] = (\exists x) (P[x] \vee H[x]).$

That is, the universal quantifier $\forall$ and the existential quantifier $\exists$ can be distributed respectively over $\wedge$ and $\vee$.

However, we must be careful about *(we have already mentioned these inequalities)*

- (vii) $(\forall x) E[x] \vee (\forall x) H[x] \neq (\forall x) (P[x] \vee H[x])$ and
- (viii) $(\exists x) P[x] \wedge (\exists x) H[x] \neq (\exists x) (P[x] \wedge H[x])$

Steps for Transforming an FOPL Formula into Prenex Normal Form

Step 1: Remove the connectives ' $\leftrightarrow$ ' and ' $\rightarrow$ ' using the equivalences

$$P \leftrightarrow G = (P \rightarrow G) \wedge (G \rightarrow P)$$

$$P \rightarrow G = \sim P \vee G$$

Step 2: Use the equivalence to remove an even number of $\sim$'s

$$\sim (\sim P) = P$$

Step 3: Apply De Morgan's laws in order to bring the negation signs immediately before atoms.

$$\sim (P \vee G) = \sim P \wedge \sim G$$

$$\sim (P \wedge G) = \sim P \vee \sim G$$

and the quantification laws

$$\sim ((\forall x) P[x]) = (\exists x) (\sim P[x])$$

$$\sim ((\exists x) P[x]) = (\forall x) (\sim P[x])$$

Step 4: Rename bound variables **if necessary**

Step 5: Bring quantifiers to the left before any predicate symbol appears in the formula. This is achieved by using (i) to (vi) discussed above.

We have already discussed that, if all occurrences of a bound variable are replaced uniformly throughout by another variable not occurring in the formula, then the equivalence is preserved. Also, we mentioned under (vii) that $\forall$ does not distribute over $\wedge$ and under (viii) that $\exists$ does not distribute over $\vee$. In such cases, in order to bring quantifiers to the left of the rest of the formula, we may have to first rename one of bound variables, say x , may be renamed as z , which does not occur either as free or bound in the other component formulas. And then we may use the following equivalences.

$$(Q1 x) P[x] \vee (Q2 x) H[x] = (Q1 x) (Q2 z) (P[x] \vee H[z])$$

$$(Q3 x) P[x] \wedge (Q4 x) H[x] = (Q3 x) (Q4 z) (P[x] \wedge H[z])$$

Example: Transform the following formulas into prenex normal forms:

$$(i) (\forall x) (Q(x) \rightarrow (\exists x) R(x, y))$$

$$(ii) (\exists x) (\sim (\exists y) Q(x, y) \rightarrow ((\exists z) R(z) \rightarrow S(x)))$$

$$(iii) (\forall x) (\forall y) ((\exists z) Q(z, y, z) \wedge ((\exists u) R(x, u) \rightarrow (\exists v) R(y, v))).$$

Part (i)

Step 1: By removing ' $\rightarrow$ ', we get

$$(\forall x) (\sim Q(x) \vee (\exists x) R(x, y))$$

Step 2: By renaming x as z in $(\exists x) R(x, y)$ the formula becomes

$$(\forall x) (\sim Q(x) \vee (\exists z) R(z, y))$$

Step 3: As $\sim Q(x)$ does not involve z , we get

$$(\forall x) (\exists z) (\sim Q(x) \vee R(z, y))$$

Part (ii)

$$(\exists x) (\sim (\exists y) Q(x, y) \rightarrow ((\exists z) R(z) \rightarrow S(x)))$$

Step 1: Removing outer ' $\rightarrow$ ' we get

$$(\exists x) (\sim (\sim ((\exists y) Q(x, y))) \vee ((\exists z) R(z) \rightarrow S(x)))$$

Step 2: Removing inner ' $\rightarrow$ ', and simplifying $\sim(\sim())$ we get

$$(\exists x) ((\exists y) Q(x, y) \vee (\sim (\exists z) R(z)) \vee S(x))$$

Step 3: Taking ' $\sim$ ' innermost, we get

$$(\exists x) (\exists y) Q(x, y) \vee ((\forall z) \sim R(z) \vee S(x))$$

As the first component formula $Q(x, y)$ does not involve z , and $S(x)$ does not involve both y and z and $\sim R(z)$ does not involve y . Therefore, we may take out $(\exists y)$ and $(\forall z)$ so that we get

$(\exists x) (\exists y) (\forall z) (Q(x, y) \vee (\sim R(z) \vee S(x)))$, which is the required formula in prenex normal form.

Part (iii)

$$(\forall x) (\forall y) ((\exists z) Q(x, y, z) \wedge ((\exists u) R(x, u) \rightarrow (\exists v) R(y, v)))$$

Step 1: Removing ' $\rightarrow$ ', we get

$$(\forall x) (\forall y) ((\exists z) Q(x, y, z) \wedge (\sim ((\exists u) R(x, u)) \vee (\exists v) R(y, v)))$$

Step 2: Taking ' $\sim$ ' innermost, we get

$$(\forall x) (\forall y) ((\exists z) Q(x, y, z) \wedge ((\forall u) \sim R(x, u) \vee (\exists v) R(y, v)))$$

Step 3: As the variables z , u , and v do not occur outside the formulas within their respective scopes, therefore, we can take all quantifiers outside, preserving the order of their occurrences, we get

$$(\forall x) (\forall y) (\exists z) (\forall u) (\exists v) (Q(x, y, z) \wedge (\sim R(x, u) \vee R(y, v)))$$

Skolemization: A further refinement of Prenex Normal Form (PNF) called (Skolem) Standard Form, is the basis of problem solving through the Resolution Method. The Resolution Method will be discussed next.

The **Standard Form of a formula of FOPL** is obtained through the following three steps:

- (1) The given formula should be converted to Prenex Normal Form (PNF), and then
- (2) Convert the Matrix of the PNF, i.e, the quantifier-free part of the PNF, into conjunctive normal form
- (3) Skolemization: Eliminate the existential quantifiers using Skolem constants and functions

Before illustrating the process of conversion of a formula of FOPL to Standard Normal Form, through examples, we discuss Skolem functions briefly.

Skolem Function

As mentioned earlier that $(\exists x) (\forall y) P(x,y) \neq (\forall y) (\exists x) P(x,y) \dots\dots(1)$

For example, if $P(x,y)$ stands for the relation ' $x > y$ ' in the set of integers, then the L.H.S. of the inequality (i) above states: *some (fixed) integer (x) is greater than all integers (y)*. This statement is False.

On the other hand, R.H.S. of the inequality (1) states: *for each integer y, there is an integer x so that $x > y$* . This statement is True.

The difference in meaning of the two sides of the inequality arises because of the fact that on L.H.S. x in $(\exists x)$ is independent of y in $(\forall y)$ **whereas** on R.H.S x is dependent on y . In other words, x on L.H.S. of the inequality can be replaced by some constant, say ' c ', whereas on the right-hand side x is some function, say, $f(y)$ of y .

Therefore, the two parts of the inequality (i) above may be written as

L.H.S. of (1) = $(\exists x) (\forall y) P(x,y) = (\forall y) P(c,y)$,

Dropping x because there is no x appearing in $(\forall y) P(c,y)$

R.H.S. of (1) = $(\forall y) (\exists x) P(f(y),y) = (\forall y) P(f(y), y)$

The above argument, in essence, explains what is meant by each of the terms viz., *skolem constant, skolem function and Skolemization*.

The constants and functions that replace existential quantifiers are respectively called **Skolem constants and Skolem functions**. The process of replacing all existential variables by Skolem constants and variables is called **Skolemization**.

A formula that is obtained after applying the steps for

- (i) reduction to PNF and then to
- (ii) CNF and then
- (iii) Applying Skolemization is called **Skolem Standard Form** or just **Standard Form**.

We explain through examples the Skolemization process after PNF and CNF have already been obtained.

Example: Skolomize the following:

(i) $(\exists x_1) (\exists x_2) (\forall y_1) (\forall y_2)(\exists x_3)(\forall y_3) P(x_1, x_2, x_3, y_1, y_2, y_3)$

(ii) $(\exists x_1)(\forall y_1)(\exists x_2)(\forall y_2) (\exists x_3)P(x_1, x_2, x_3, y_1, y_2) \wedge (\exists x_1)(\forall y_3)(\exists x_2) (\forall y_4)Q(x_1, x_2, y_3, y_4)$

Solution (i) As existential quantifiers x_1 and x_2 precede all universal quantifiers, therefore, x_1 and x_2 are to be replaced by *constants*, but by distinct

constants, say by 'c' and 'd' respectively. As existential variable x_3 is preceded by universal quantifiers y_1 and y_2 , x_3 is replaced by some function $f(y_1, y_2)$ of the variables y_1 and y_2 . After making these substitutions and dropping universal and existential variables, we get the Skolemized form of the given formula as

$$(\forall y_1) (\forall y_2) (\forall y_3) (c, d, f(y_1, y_2), y_1, y_2, y_3).$$

Solution (ii) As a first step we must bring all the quantifications in the beginning of the formula through Prenex Normal Form reduction. Also,

$$(\exists x) \dots P(x, \dots) \wedge (\exists x) \dots Q(x, \dots) \neq (\exists x) (\dots P(x) \wedge \dots Q(x, \dots)),$$

therefore, we rename the second occurrences of quantifiers $(\forall x_1)$ and $(\forall x_2)$ by renaming these as x_5 and x_6 . Hence, after renaming and pulling all the quantifiers to the left, we get

$$(\exists x_1) (\forall y_1) (\exists x_2) (\forall y_2) (\exists x_3) (\exists x_5) (\forall y_3) (\exists x_6) (\forall y_4) \\ (P(x_1, x_2, x_3, y_1, y_2) \wedge Q(x_5, x_6, y_3, y_4))$$

Then the existential variable x_1 is independent of all the universal quantifiers. Hence, x_1 may be replaced by a constant say, 'c'. Next x_2 is preceded by the universal quantifier y_1 hence, x_2 may be replaced by $f(y_1)$. The existential quantifier x_3 is preceded by the universal quantifiers y_1 and y_2 . Hence x_3 may be replaced by $g(y_1, y_2)$.

The existential quantifier x_5 is again preceded by universal quantifier y_1 and y_2 . In other words, x_5 is also a function of y_1 and y_2 . But, we have to use a different function symbol say h and replace x_5 by $h(y_1, y_2)$. Similarly x_6 may be replaced by

$$j(y_1, y_2, y_3).$$

Thus, (Skolem) Standard Form becomes

$$(\forall y_1) (\forall y_2) (\forall y_3) (P(c, f(y_1), g(y_1, y_2), y_1, y_2) \wedge Q(h(y_1, y_2), j(y_1, y_2, y_3))).$$

Check Your Progress -2

Ex: 4 (i) Transform the formula $(\forall x) P(x) \rightarrow (\exists x) Q(x)$ into prenex normal form.

(ii) Obtain a prenex normal form for the formula

$$(\forall x) (\forall y) ((\exists z) (P(x, y) \wedge P(y, z)) \rightarrow (\exists u) Q(x, y, u))$$

Ex 5. Obtain a (skolem) standard form for each of the following formula:

- (i) $(\exists x) (\forall y) (\forall v) (\exists z) (\forall w) (\exists u) P(x, y, z, u, v, w)$
- (ii) $(\forall x) (\exists y) (\exists z) ((P(x, y) \vee \sim Q(x, z)) \rightarrow R(x, y, z))$

5.7 RESOLUTION & UNIFICATION

In the beginning of the previous section, we mentioned that resolution method for FOPL requires discussion of a number of complex new concepts. Also, ,

we discussed (Skolem) Standard Form and also discussed how to obtain Standard Form for a given formula of FOPL. In this section, along with resolution, we will introduce two new, and again complex, concepts, viz., *substitution and unification*.

The complexity of the resolution method for FOPL mainly results from the fact that a clause in FOPL is generally of the form : $P(x) \vee Q(f(x), x, y) \vee \dots$, in which the variables x, y, z , may assume any one of the values of their domain.

Thus, the atomic formula $(\forall x) P(x)$, which after dropping of universal quantifier, is written as just $P(x)$ stands for $P(a_1) \wedge P(a_2) \dots \wedge P(a_n)$ where the set $\{a_1, a_2, \dots, a_n\}$ is assumed here to be domain (x) .

Similarly, $(\exists x) P(x)$ stands for $(P(a_1) \vee P(a_2) \vee \dots \vee P(a_n))$

However, in order to resolve two clauses – one containing say $P(x)$ and the other containing $\sim P(y)$ where x and y are universal quantifiers, possibly having some restrictions, we have to know which values of x and y satisfy both the clauses. For this purpose we need the concepts of **substitution** and **unification** as defined and discussed in the rest of the section.

Instead of giving formal definitions of substitution, unification, unifier, most general unifier, resolvent, and resolution of clauses in FOPL, we illustrate the concepts through examples and minimal definitions, if required

Example: Let us consider our old problem:

To conclude

(i) Raman is mortal

From the following two statements:

(ii) Every man is mortal and

(iii) Raman is a man

Using the notations

MAN (x) : x is a man

MORTAL (x) : x is mortal,

the problem can be formulated in symbolic logic as: Conclude

MORTAL (Raman)

from

(ii) $((\forall x) (MAN(x) \rightarrow MORTAL(x)))$

(iii) MAN (Raman).

As resolution is a refutation method, assume

(i) $\sim MORTAL (Raman)$

After Skolemization and dropping $(\forall x)$, statement (ii) in standard form becomes

- (i) $\sim \text{MAN}(x) \vee \text{MORTAL}(x)$
- (ii) $\text{MAN}(\text{Raman})$

In the above x varies over the set of human beings including Raman. Hence, one special instance of (iv) becomes

$$(vi) \sim \text{MAN}(\text{Raman}) \vee \text{MORTAL}(\text{Raman})$$

At this stage, we may observe that

(a) $\text{MAN}(\text{Raman})$ and $\text{MORTAL}(\text{Raman})$ do not contain any variables, and, hence, their truth or falsity can be determined directly. Hence, each is like a formula of PL. A formula that does not contain any variable is called **ground term** or **ground formula**.

(b) Treating $\text{MAN}(\text{Raman})$ as formula of PL and using resolution method on (v) and (vi), we conclude

$$(vii) \text{MORTAL}(\text{Raman}),$$

Resolving (i) and (vii), we get **False**. Hence, the solution.

Unification: In the process of solution of the problem discussed above, we tried to make the two expression $\text{MAN}(x)$ and $\text{MAN}(\text{Raman})$ identical. An attempt to make two or more expressions identical is called *unification*.

In order to unify $\text{MAN}(x)$ and $\text{MAN}(\text{Raman})$ identical, we found that because one of the possible values of x is *Raman* also. And, hence, we replaced x by one of its possible values : *Raman*.

This replacement of a variable like x , by a term (*which may be another variable also*)

which is one of the possible values of x , is called **substitution**. The substitution, in this case is denoted formally as $\{\text{Raman}/x\}$

Substitution, in general, **notationally** is of the form $\{t_1 / x_1, t_2 / x_2 \dots t_m / x_m\}$ where $x_1, x_2 \dots, x_m$ are variables and $t_1, t_2 \dots t_m$ are terms and t_i replaces the variable x_i in some expression.

Example: (i) Assume Lord Krishna is loved by everyone who loves someone
(ii) Also assume that no one loves nobody. Deduce Lord Krishna is loved by everyone.

Solution: Let us use the symbols

Love (x, y): x loves y (or y is loved by x)

LK : Lord Krishna

Then the given problem is formalized as :

- (i) $(\forall x) ((\exists y) \text{Love}(x, y) \rightarrow \text{Love}(x, \text{LK}))$
- (ii) $\sim (\exists x) ((\forall y) \sim \text{Love}(x, y))$

To show : $(\forall x) (\text{Love}(x, LK))$

As resolution is a refutation method, assume negation of the last statement as an axiom.

(iii) $\sim (\forall x) \text{Love}(x, LK)$

The formula in (i) above is reduced in standard form as follows:

$$\begin{aligned} & (\forall x) (\sim (\exists y) \text{Love}(x, y) \vee \text{Love}(x, LK)) \\ &= (\forall x) ((\forall y) \sim \text{Love}(x, y) \vee \text{Love}(x, LK)) \\ &= (\forall x) (\forall y) (\sim \text{Love}(x, y) \vee \text{Love}(x, LK)) \\ & (\because (\forall y) \text{ does not occurs in } \text{Love}(x, LK)) \end{aligned}$$

After dropping universal quantifications, we get

(iv) $\sim \text{Love}(x, y) \vee \text{Love}(x, LK)$

Formula (ii) can be reduced to standard form as follows:

(ii) $= (\forall x) (\exists y) \text{Love}(x, y)$
y is replaced through skolemization by $f(x)$
so that we get

$(\forall x) \text{Love}(x, f(x))$

Dropping the universal quantification

(v) $\text{Love}(x, f(x))$

The formula in (iii) can be brought in standard form as follows:

(iii) $= (\exists x) (\sim \text{Love}(x, LK))$

As existential quantifier x is not preceded by any universal quantification, therefore, x may be substituted by a constant a , i.e., we use the substitution $\{a/x\}$ in (iii) to get the standard form:

(vi) $\sim \text{Love}(a, LK)$.

Thus, to solve the problem, we have the following standard form formulas for resolution:

(iv) $\sim \text{Love}(x, y) \vee \text{Love}(x, LK)$

(v) $\text{Love}(x, f(x))$

(vi) $\sim \text{Love}(a, LK)$.

Two possibilities of resolution exist for two pairs of formulas viz.

one possibility: resolving (v) and (vi).

second possibility : resolving (iv) and (vi).

The possibilities exist because in each pair, the predicate *Love* occurs in complemented form in the respective pair.

Next we attempt to resolve (v) and (vi)

For this purpose we attempt to make the two formulas $\text{Love}(x, f(x))$ and $\text{Love}(a, LK)$ identical, through unification involving substitutions. We start from the left, matching the two formulas, term by term. First place where matching may fail is when 'x' occurs in one formula and 'a' occurs in the other formula. **As, one of these happens to be a variable**, hence, the substitution $\{a/x\}$ can be used to unify the portions so far.

Next, possible disagreement through term-by-term matching is obtained when we get the two disagreeing terms from two formulas as $f(x)$ and LK . **As neither of $f(x)$ and LK is a variable** (note $f(x)$ involves a variable but is itself not a variable), hence, no unification and, hence, no resolution of (v) and (vi) is possible.

Next, we attempt unification of **(vi) $\text{Love}(a, LK)$ with $\text{Love}(x, LK)$** of (iv).

Then first term-by-term possible disagreement occurs when the corresponding terms are 'a' and 'x' respectively. As one of these is a variable, the substitution $\{a/x\}$ unifies the parts of the formulas so far. Next, the two occurrences of LK , one each in the two formulas, match. Hence, the two formulas can be unified through the substitution $\{a/x\}$. Though the unification has been attempted in corresponding smaller parts, substitution has to be carried **in the whole of the formula**, in this case, in whole of (iv). Thus, after substitution, (iv) becomes
(viii) $\sim \text{Love}(a, y) \vee \text{Love}(a, LK)$

Resolving (viii) with (vi) we get

(ix) $\sim \text{Love}(a, y)$

In order to resolve (v) and (ix), we attempt to unify **$\text{Love}(x, f(x))$** of (v) with

$\text{Love}(a, y)$ of (ix).

The term-by-term matching leads to possible disagreement of a of (ix) with x of (v).

As, one of these is a variable, hence, the substitution $\{a/x\}$ will unify the portions considered so far.

Next, possible disagreement may occur with $f(x)$ of (v) and y of (ix). As one of these is a variable viz. y , therefore, we can unify the two terms through the substitution $\{f(x)/y\}$. Thus, the complete substitution $\{a/x, f(x)/y\}$ is required to match the formulas. After making the substitutions, we get (v) becomes $\text{Love}(a, f(x))$ and (ix) becomes $\sim \text{Love}(a, f(x))$

Resolving these formulas we get **False**. Hence, the proof.

Check you Progress - 3

Ex. 6: Unify, if possible, the following three formulas:

- (i) $Q(u, f(y, z)),$
- (ii) $Q(u, a)$
- (iii) $Q(u, g(h(k(u))))$

Ex. 7: Determine whether the following formulas are unifiable or not:

- (i) $Q(f(a), g(x))$
- (ii) $Q(x, y)$

Example: Find resolvents, if possible for the following pairs of clauses:

- (i) $\sim Q(x, z, x) \vee Q(w, z, w)$ and
- (ii) $Q(w, h(v, v), w)$

Solution: As two literals with predicate Q occur and are mutually negated in (i) and (ii), therefore, there is possibility of resolution of $\sim Q(x, z, x)$ from (i) with $Q(w, h(v, v), w)$ of (ii). We attempt to unify $Q(x, z, x)$ and $Q(w, h(v, v), w)$, if possible, by finding an appropriate substitution. First terms x and w of the two are variables, hence, unifiable with either of the substitutions $\{x/w\}$ or $\{w/x\}$. Let us take $\{w/x\}$.

Next pair of terms from the two formulas, viz, z and $h(v, v)$ are also unifiable, because, one of the terms is a variable, and the required substitution for unification is $\{h(v, v)/z\}$.

Next pair of terms at corresponding positions is again $\{w, x\}$ for which, we have determined the substitution $\{w/x\}$. Thus, the substitution $\{w/x, h(v, v)/z\}$ unifies the two formulas. Using the substitutions, (i) and (ii) become resp. as

- (iii) $\sim Q(w, h(v, v), w) \vee Q(w, h(v, v), w)$
- (iv) $Q(w, h(v, v), w)$

Resolving, we get

$Q(w, h(v, v), w),$

which is the required resolvent.

5.8 SUMMARY

In this unit, initially, we discuss how PL is inadequate to solve even simple problems and requires some extension of PL or some other formal inferencing system to compensate for this inadequacy. First Order Predicate Logic (FOPL) is such an extension of PL that is discussed in the unit.

Next, the syntax of the proper structure of a formula of FOPL is discussed. In this respect, a number of new concepts including those of quantifiers, variables, constants, terms, free and bound occurrences of variables, closed and open wffs, consistency/validity of wffs, etc., are introduced.

Next, two normal forms viz. Prenex Normal Form (PNF) and Skolem Standard Normal Form are introduced. Finally, tools and techniques developed in the unit, are used to solve problems involving logical reasoning.

5.9 SOLUTIONS/ANSWERS

Check Your Progress - 1

Ex. 1 (i) $(\forall x) (P(x) \rightarrow Q(x))$

(ii) $(\exists x) (P(x) \wedge Q(x))$

(iii) $\sim (\forall x) (Q(x) \rightarrow P(x))$

Ex. 2

(i) There is (at least) one (person) who is a used-car dealer.

(ii) There is (at least) one (person) who is honest.

(iii) All used-car dealers are dishonest.

(iv) (At least) one used-car dealer is honest.

(v) There is at least one thing in the universe, (for which it can be said that) if that something is Honest then that something is a used-car dealer

Note: the above translation is not the same as: Some no gap one honest, is a used-car dealer.

Ex 3: (i) After removal of ' $\rightarrow$ ' we get the given formula

$$= \sim P(a) \vee \sim ((\exists x) P(x))$$

$$= \sim P(a) \vee (\forall x) (\sim P(x))$$

Now $P(a)$ is an atom in PL which may assume any value T or F. On taking $P(a)$ as F the given formula becomes T, hence, consistent.

(ii) The formula can be written

$(\forall x) P(x) \vee \sim (\forall x) (P(x))$, by taking negation outside the second disjunct and then renaming.

The $(\forall x) P(x)$ being closed is either T or F and hence can be treated as formula of PL.

Let $\forall x P(x)$ be denoted by Q. Then the given formula may be denoted by $Q \vee \sim Q = \text{True (always)}$ Therefore, formula is valid.

Check Your Progress - 2

Ex: 4 (i) $(\forall x) P(x) \rightarrow (\exists x) Q(x) = \sim ((\forall x) P(x)) \vee (\exists x) Q(x)$ (by removing the connective $\rightarrow$)

$$= (\exists x) (\sim P(x)) \vee (\exists x) Q(x) \text{ (by taking '}' inside)}$$

$$= (\exists x) (\sim P(x) \vee Q(x)) \text{ (By taking distributivity of } \exists x \text{ over } \vee)$$

Therefore, a prenex normal form of $(\forall x) P(x) \rightarrow (\exists x) Q(x)$ is $(\exists x) (\sim P(x) \vee Q(x))$.

(ii) $(\forall x) (\forall y) ((\exists z) (P(x, y) \wedge P(y, z)) \rightarrow (\exists u) Q(x, y, u))$ (removing the connective $\rightarrow$)

$$= (\forall x) (\forall y) (\sim ((\exists z) (P(x, z) \wedge P(y, z))) \vee (\exists u) Q(x, y, u)) \quad (\text{using De Morgan's Laws})$$

$$= (\forall x) (\forall y) ((\forall z) (\sim P(x, z) \vee \sim P(y, z)) \vee (\exists u) Q(x, y, u))$$

$$= (\forall x) (\forall y) (\forall z) (\sim P(x, z) \vee \sim P(y, z) \vee Q(x, y, u)) \quad (\text{as } z \text{ and } u \text{ do not occur in the rest of the formula except their respective scopes})$$

Therefore, we obtain the last formula as a prenex normal form of the first formula.

Ex 5 (i) In the given formula $(\exists x)$ is not preceded by any universal quantification. Therefore, we replace the variable x by a (skolem) constant c in the formula and drop $(\exists x)$.

Next, the existential quantifier $(\exists z)$ is preceded by two universal quantifiers viz., v and y . we replace the variable z in the formula, by some function, say, $f(v, y)$ and drop $(\exists z)$. Finally, existential variable $(\exists u)$ is preceded by three universal quantifiers, viz., $(\forall y)$, $(\forall y)$ and $(\forall w)$. Thus, we replace in the formula the variable u by, some function $g(y, v, w)$ and drop the quantifier $(\exists u)$. Finally, we obtain the standard form for the given formula as $(\forall y) (\forall v) (\forall w) P(x, y, z, u, v, w)$

(ii) First of all, we reduce the matrix to CNF.

$$\begin{aligned} &= (P(x, y) \vee \sim Q(x, z)) \rightarrow R(x, y, z) \\ &= (\sim P(x, y) \wedge Q(x, z)) \vee R(x, y, z) \\ &= (\sim P(x, y) \vee R(x, y, z)) \wedge (Q(x, z) \vee R(x, y, z)) \end{aligned}$$

Next, in the formula, there are two existential quantifiers, viz., $(\exists y)$ and $(\exists z)$. Each of these is preceded by the only universal quantifier, viz. $(\forall x)$.

Thus, each variable y and z is replaced by a function of x . But the two functions of x for y and z must be different functions. Let us assume, variable, y is replaced in the formula by $f(x)$ and the variable z is replaced by $g(x)$. Thus the initially given formula, after dropping of existential quantifiers is in the standard form:

$$(\forall x) ((\sim P(x, y) \vee R(x, y, z)) \wedge (Q(x, z) \vee R(x, y, z)))$$

Check Your Progress - 3

Ex 6 : Refer to section 5.7

Ex 7 : Refer to section 5.7

5.10 FURTHER READINGS

1. Ela Kumar, “ Artificial Intelligence”, IK International Publications
2. E. Rich and K. Knight, “Artificial intelligence”, Tata Mc Graw Hill Publications
3. N.J. Nilsson, “Principles of AI”, Narosa Publ. House Publications
4. John J. Craig, “Introduction to Robotics”, Addison Wesley publication
5. D.W. Patterson, “Introduction to AI and Expert Systems" Pearson publication
6. McKay, Thomas J., *Modern Formal Logic* (Macmillan Publishing Company, 1989).
7. Gensler, Harry J. *Symbolic Logic: Classical and Advanced Systems* (Prentice Hall of India, 1990).
8. Klenk, Virginia *Understanding Symbolic Logic* (Prentice Hall of India 1983)
9. Copi Irving M. & Cohen Carl, *Introduction Logic IX edition*, (Prentice Hall of India, 2001).



ignou
THE PEOPLE'S
UNIVERSITY

UNIT 6 RULE BASED SYSTEMS AND OTHER FORMALISM

Structure	Page Nos.
6.0 Introduction	213
6.1 Objectives	214
6.2 Rule Based Systems	215
6.2.1 Forward chaining	217
6.2.2 Backward chaining	220
6.2.3 Conflict resolution	224
6.3 Semantic nets	226
6.4 Frames	228
6.5 Scripts	229
6.6 Summary	231
6.7 Solutions/Answers	231
6.8 Further/Readings	232

6.0 INTRODUCTION

Computer Science is the study of how to create models that can be represented in and executed by some computing equipment. In this respect, the task of a computer scientist is to create, in addition to a model of the problem domain, a model of a domain expert who is highly skilled in solving problems in the domain under consideration, and the concerned field is related to the field of Expert Systems.

First of all, we must understand that an expert system is nothing but a computer program or a set of computer programs that contains the knowledge and inference capability of an expert, most generally a human expert, in a particular domain. An expert system is supposed to contain the capability to lead to some conclusion, based on the inputs provided; the system already contains some pre-existing information, which is processed to infer some conclusion. An expert system belongs to the branch of Computer Science called Artificial Intelligence.

Taking into consideration all the points discussed above, one of the many possible definitions of an Expert System is: *“An Expert System is a computer program that possesses or represents knowledge in a particular domain, has the capability of processing/ manipulating or reasoning with this knowledge with a view to solving a problem, giving some achievement or achieving some specific goal.”*

Whereas, the *Artificial Intelligence* programs written to achieve expert-level competence in solving problems in different domains are more commonly called **knowledge-based systems**. A **knowledge-based system** is any system that performs a job or task by applying rules of thumb to a symbolic representation of knowledge, instead of employing mostly algorithmic or statistical methods. Often, the term *expert systems* is reserved for programs whose knowledge base contains the knowledge used by human experts, in contrast to knowledge gathered from textbooks or non-experts. **But more often than not, the two terms, expert systems and knowledge-based systems, are taken as synonyms.** Together, they represent the most widespread type of *AI* application.

One of the underlying assumptions in Artificial Intelligence is that **intelligent behaviour can be achieved through the manipulation of symbolic structures** (representing bits of knowledge). One of the main issues in *AI* is to find appropriate representations of problem elements and available actions as symbol structures so that the representation can be used to intelligently solve problems. In *AI*, an important criterion for knowledge representation schemes or languages is that they should support *inference*. For intelligent action, the inferencing capability is essential in view of the fact that we can't represent explicitly everything that the system might ever need to know—some things have to be left implicit, to be inferred/deduced by the system as and when needed in problem solving.

In general, a good knowledge representation scheme should have the following features:

- It should allow us to express the knowledge we wish to represent in the language. For example, the mathematical statement: *Every symmetric and transitive relation on a domain need not be reflexive* is not expressible in First Order Logic.
- It should allow new knowledge to be inferred from a basic set of facts, as discussed above.
- It should have well-defined *syntax* and *semantics*.

Building an expert system is known as knowledge engineering, and its practitioners are called knowledge engineers. It is the job of the knowledge engineer to ensure that the computer has all the knowledge needed to solve a problem. The knowledge engineer must choose one or more forms to represent the required knowledge, i.e., s/he must choose one or more knowledge representation schemes.

Several knowledge representing schemes like predicate logic, semantic nets, frames, scripts and rule based systems, exists; and we will discuss them in this unit. Some popular knowledge representation schemes are:

- *First order logic,*
- *Semantic networks,*
- *Frames,*
- *Scripts, and*
- *Rule-based systems.*

As predicate logic has been discussed in previous blocks, we will discuss the remaining knowledge representation schemes in this unit.

6.1 OBJECTIVES

After going through this unit, you should be able to:

- understand the basics of an expert system
- understand the basics of Knowledge based systems

- discuss the various knowledge representation schemes like rule-based systems, semantic nets, frames, and scripts

6.2 RULE BASED SYSTEMS

We know that **planning** is the process that exploits the structure of the problem under consideration for designing a sequence of actions in order to solve it.

In order to plan a solution to the problem, one should have the knowledge of the nature and structure of the problem domain under consideration. For the purpose of planning, the problem environments are divided into two categories, viz., classical planning environments and non-classical planning environments. The **classical** planning environments/domains are fully observable, deterministic, finite, static and discrete. On the other hand, **non-classical** planning environments may be only partially observable and/or stochastic. In this unit, we discuss planning only for classical environments.

Let's begin with the Rule-Based Systems:

Rather than representing knowledge in a declarative and somewhat static way (as a set of statements, each of which is true), rule-based systems represent knowledge in terms of a set of rules each of which specifies the conclusion that could be reached under given conditions or in different situations. A rule-based system consists of

- (i) Rule base, which is a set of IF-THEN *rules*,
- (ii) A bunch of *facts*, and
- (iii) Some *interpreter* of the facts and rules, which is a mechanism that decides which rule to apply based on the set of available facts. The interpreter also initiates the action suggested by the rule selected for application.

A Rule-base may be of the form:

R₁: If A is an animal and A barks, then A is a dog

F1: Rocky is an animal

F2: Rocky Barks

The rule-interpreter, after scanning the above rule-base, may conclude: Rocky is a dog.

After this interpretation, the rule base becomes

R₁: If A is an animal and A barks, then A is a dog

F1: Rocky is an animal

F2: Rocky barks

F3: Rocky is a dog.

There are two broad kinds of rule-based systems:

- **Forward chaining systems,**
- **Backward chaining systems.**

In a **forward** chaining system, we start with the initial facts, and keep using the rules to draw new intermediate conclusions (or take certain actions) given those facts. The process terminates when the final conclusion is established. In a **backward** chaining system, we start with some goal statements, which are intended to be established, and keep looking for rules that would allow us to reach a conclusion, setting new sub-goals in the process of reaching the ultimate goal. In the next round, the subgoals become the new goals to be established. The process terminates when in this process all the subgoals are given as facts. Forward chaining systems are primarily **data-driven**, while backward chaining systems are **goal-driven**. We will discuss each in detail. *Next, we discuss in detail some of the issues involved in a rule-based system.*

Advantages of Rule-base

A basic principle of a rule-based system is that each rule is an independent piece of knowledge. In an IF-THEN rule, the IF-part contains all the conditions for the application of the rule under consideration. THEN-part tells the action to be taken by the interpreter. The interpreter need not search anywhere else except within the rule itself for the conditions required for application of the rule.

Another important consequence of the above-mentioned characteristic of a rule-based system is that no rule can call upon any other, and hence, rules are ignorant and independent of each other. This gives a highly modular structure to the rule-based systems. Because of the highly modular structure of the rule-base, the rule-based system, the addition, deletion, and modification of a rule can be done without any side effects.

Disadvantages

The main problem with the rule-based system is that when the rule base grows and becomes very large, checking (i) whether a new rule intended to be added is redundant, i.e., whether it is already covered by some of the earlier rules, becomes difficult. Still worse, as the rule-base grows, checking the consistency of the rule-base also becomes quite difficult. By consistency, we mean that there may be two rules having similar conditions, but the actions of the two rules **conflict** with each other.

Let us first define working memory before we study forward and backward chaining systems.

Working Memory: A working memory is a representation, in which

- lexically, there are application-specific symbols.
- structurally, assertions are a list of application-specific symbols.
- semantically, assertions denote facts.
- assertions can be added or removed from working memory.

Rule-based systems usually work in domains where conclusions are rarely certain, even when we are careful enough to try and include everything we can think of in the antecedent or condition parts of rules.

Sources of Uncertainty

Two important sources of uncertainty in rule-based systems are:

- ✓ The theory of the domain may be vague or incomplete, so the methods to generate exact or accurate knowledge are not known.
- ✓ Case data may be imprecise or unreliable and evidence may be missing or in conflict.

So even though methods to generate exact knowledge are known, they are impractical due to lack of data, imprecision or data or problems related to data collection.

So, *rule-based deduction system developers often build some sort of certainty or probability computing procedure over and above the normal condition-action format of rules.* Certainty computing procedures attach a **probability between 0 and 1** to each assertion or fact. Each probability reflects how certain an assertion is, whereas a certainty factor of 0 indicates that the assertion is definitely false and a certainty factor of 1 indicates that the assertion is definitely true.

Example 1: In the example discussed above, the assertion (Ram at-home) may have a certainty factor, say 0.7, attached to it.

Example 2: In MYCIN, a rule-based expert system (which we will discuss later), a rule in which statements that link evidence to hypotheses are expressed as decision criteria, may look like:

IF patient has symptoms s1, s2, s3 and s4
AND certain background conditions t1, t2 and t3 hold
THEN the patient has disease d6 with certainty 0.75

For a detailed discussion on certainty factors, the reader may refer to probability theory, fuzzy sets, possibility theory, Dempster-Shafer Theory, etc.

6.2.1 Forward Chaining Systems

In a forward chaining system, the facts in the system are represented in a **working memory**, which is continually updated, so on the basis of a rule that is currently being applied, the number of facts may either increase or decrease. Rules in the system represent possible actions to be taken when specified conditions hold on items in the working memory—they are sometimes called **condition-action or antecedent-consequent rules**. The conditions are usually **patterns** that must *match* items in the working memory, while the actions usually involve **adding or deleting** items from the working memory. So **we can say that forward chaining proceeds forward, beginning with facts, chaining through rules, and finally establishing the goal.** Forward chaining

systems usually represent rules in standard implicational form, with an antecedent or condition part consisting of positive literals, and a consequent or conclusion part consisting of a positive literal.

The interpreter controls the application of the rules, given the working memory, thus controlling the system's activity. It is based on a cycle of activity sometimes known as a **recognize-act** cycle. The system first checks to find all the rules whose condition parts are satisfied, i.e., those rules which are applicable, given the current state of working memory, **a rule is applicable if** each of the literals in its antecedent, i.e., the condition part, can be unified with a corresponding fact using consistent substitutions. This restricted form of unification is called pattern matching. It then selects one and performs the actions in the action part of the rule, which may involve the addition or deletion of facts. The actions will result in a new, i.e., updated working memory, and the cycle starts again (**when more than one rule is applicable**, then some sort of external **conflict resolution scheme** is used to decide which rule will be applied. But when there are a large number of rules and facts, then the number of unifications that must be tried becomes prohibitive or difficult). This cycle will be repeated until either there is no rule that fires, or the required goal is reached.

Rule-based systems vary greatly in their details and syntax. Let us take the following example in which we use forward chaining :

Example

Let us assume that the working memory initially contains the following facts:

(day monday)

(at-home ram)

(does-not-like ram)

Let the existing set of rules are:

R1 : IF (day monday)
THEN ADD to working memory the fact : (working-with ram)

R2 : IF (day monday)
THEN ADD to working memory the fact : (talking-to ram)

R3 : IF (talking-to X) AND (working-with X)
THEN ADD to working memory the fact : (busy-at-work X)

R4 : IF (busy-at-work X) OR (at-office X)
THEN ADD to working memory the fact : (not-at-home X)

R5 : IF (not-at-home X)
THEN DELETE from working memory the fact : (happy X)

R6 : IF (working-with X)
THEN DELETE from working memory the fact : (does-not-like X)

Now, **to start the process of inference through forward chaining**, the rule-based system will first search for all the rule/s whose antecedent part/s are satisfied by the current set of facts in the working memory. *For example, in this example, we can see that the rules R1 and R2 are satisfied, so the system will choose one of them using its **conflict resolution strategies**.* Let rule R1 is chosen. So (working with ram) is added to the working memory (after substituting “ram” in place of X). So working memory now looks like:

(working with RAM)
(day monday)
(at-home ram)
(does-not-like ram)

Now this cycle begins again, the system looks for rules that are satisfied, and it finds rules R2 and R6. Let the system choose rule R2. So now (taking-to ram) is added to working memory. So now working memory contains the following:

(talking-to Ram)
(working with RAM)
(day monday)
(at-home ram)
(does-not-like ram)

Now in the next cycle, rule R3 fires, so now (busy-at-work ram) is added to working memory, which now looks like:

(busy-at-work ram)
(talking-to ram)
(working-with ram)
(day monday)
(at-home ram)
(does-not-like ram)

Now, the antecedent parts of rules R4 and R6 are satisfied. Let rule R4 fire, so (not-at-home, ram) is added to working memory, which now looks like :

(not-at-home ram)
(busy-at-work ram)
(talking-to ram)
(working-with ram)
(day monday)
(at-home ram)
(does-not-like ram)

In the next cycle, rule R5 fires, so (at-home ram) is removed from the working memory :

(not-at-home ram)
(busy-at-work ram)
(talking-to ram)
(working-with ram)
(day monday)
(does-not-like ram)

The forward chaining will continue like this. But we have to be sure of one thing, that the ordering of the rules firing is important. A change in the ordering sequence of rule firing may result in a different working memory.

6.2.2 Backward Chaining Systems

In forward chaining systems, we have seen how rule-based systems are used to draw new conclusions from existing data and then add these conclusions to a working memory. **The forward chaining approach is most useful when** we know all the initial facts, but do not have much idea what the conclusion might be.

If we know what the conclusion would be, or have some specific hypothesis to test, forward chaining systems may be inefficient. In forward chaining, we keep on moving ahead until no more rules apply or we have added our hypothesis to the working memory. But in the process, the system is likely to do a lot of additional and irrelevant work, adding uninteresting or irrelevant conclusions to the working memory. Let us say that in the example discussed before, suppose we want to find out whether “Ram is at home”. We could repeatedly fire rules, updating the working memory, checking each time whether **(at-home ram)** is found in the new working memory. But maybe we had a whole batch of rules for drawing conclusions about what happens when I’m working, or what happens on Monday—we really don’t care about this, so we would rather only have to draw the conclusions that are relevant to the goal.

This can be done by **backward chaining** from the goal state or from some hypothesized state in which we are interested. This is essentially how Prolog works. Given a goal state to prove, for example, **(at-home ram)**, the system will first check whether the goal matches the initial facts. If it does, that goal succeeds. If it doesn’t, the system will look for rules whose conclusions, i.e., *actions*, match the goal. One such rule will be chosen, and the system will then try to prove any facts in the preconditions of the rule using the same procedure, setting these as new goals to prove. **We should note that a backward chaining system does not need to update a working memory.** Instead, it needs to keep track of the goals that must be proved to establish the main hypothesis. So, we can say that **in a backward chaining system, the reasoning proceeds “backward”, beginning with the goal to be established, chaining through rules, and finally anchoring in facts.**

Although in principle, the same set of rules can be used for both forward and backward chaining, in practice, we may choose to write the rules slightly differently for backward chaining. In backward chaining, we are concerned with matching the conclusion of a rule against some goal that we are trying to prove. So, the ‘then or consequent’ part of the rule is usually not expressed as an action to take (e.g., add/delete), but as a state that will be true if the premises are true.

To learn more, let us consider a different example in which backward chaining is used. In this example, the system identifies an animal based on its properties stored in the working memory:

Example

1. Let us assume that the working memory initially contains the following facts:

(has-hair raja)	representing the fact “raja has hair”
(big-mouth raja)	representing the fact “raja has a big mouth”
(long-pointed-teeth raja)	representing the fact “raja has long pointed teeth”
(claws raja)	representing the fact “raja has claws”

Let the existing set of rules are:

1. IF (gives-milk X)
THEN (mammal X)
2. IF (has-hair X)
THEN (mammal X)
3. IF (mammal X) AND (eats-meat X)
THEN (carnivorous X)
4. IF (mammal X) AND (long-pointed-teeth X) AND (claws X)
THEN (carnivorous X)
5. IF (mammal X) AND (does-not-eat-meat X)
THEN (herbivorous X)
6. IF (carnivorous X) AND (dark-spots X)
THEN (cheetah, X)
7. IF (herbivorous X) AND (long-legs X) AND (long-neck X) AND (dark-spots X)
THEN (giraffe, X)
8. IF (carnivorous X) AND (big-mouth X)
THEN (lion, X)
9. IF (herbivorous X) AND (long-trunk X) AND (big-size X)
THEN (elephant, X)
10. IF (herbivorous, X) AND (white-color X) AND ((black-strips X)
THEN (zebra, X)

Now, to start the process of inference through backward chaining, the rule-based system will first form a hypothesis, and then it will use the antecedent–consequent rules (previously called condition – action rules) to work backward toward hypothesis supporting assertions or facts.

Let us take the initial hypothesis that “raja is a lion” and then reason about whether this hypothesis is viable using backward chaining approach explained below:

- The system searches for a rule, which has the initial hypothesis in the consequent part that someone, i.e., raja is a lion, which it finds in rule 8.
- The system moves from consequent to antecedent part of rule 8 and it finds the first condition, i.e., the first part of antecedent which says that “raja must be a carnivorous”.
- Next, the system searches for a rule whose consequent part declares that someone i.e., “raja is a carnivorous”, two rules are found i.e., rule 3 and rule 4. We assume that the system tries rule 3 first.
- To satisfy the consequent part of rule 3, which has now become the system’s new hypothesis, the system moves to the first part of antecedent, which says that X i.e., raja has to be mammal.
- So a new sub-goal is created in which the system has to check that “raja is a mammal”. It does so by hypothesizing it and tries to find a rule having a consequent that someone or X is a mammal. Again, the system finds two rules, rule 1 and rule 2. Let us assume that the system tries rule 1 first.
- In rule 1, the system now moves to the first antecedent part, which says that X i.e., raja must give milk for it to be a mammal. The system cannot tell this because this hypothesis is neither supported by any of the rules and also it is not found among the existing facts in the working memory. So the system abandons rule 1 and tries to use rule 2 to establish that “raja is a mammal”.
- In rule 2, it moves to the antecedent, which says that X i.e., raja must have hair for it to be a mammal. The system already knows this, as it is one of the facts in working memory. So the antecedent part of rule 2 is satisfied, and so the consequent that “raja is a mammal” is established.
- Now the system backtracks to rule 3, whose first antecedent part is satisfied. In the second condition of the antecedent, it finds its new sub-goal and, in turn, forms a new hypothesis that X, i.e., raja eats meat.
- The system tries to find a supporting rule or an assertion in the working memory that says that “raja eats meat” but it finds none. So, the system abandons rule 3 and tries to use rule 4 to establish that “raja is carnivorous”.
- In rule 4, the first part of the antecedent says that raja must be a mammal for it to be carnivorous. The system already knows that “raja is a mammal” because it was already established when trying to satisfy the antecedents in rule 3.
- The system now moves to the second part of antecedent in rule 4 and finds a new sub-goal in which the system must check that X i.e., raja has long-pointed-teeth which now becomes the new hypothesis. This is already

established as “raja has long-pointed-teeth” is one of the assertions of the working memory.

- In third part of antecedent in rule 4 the system’s new hypothesis is that “raja has claws”. This also is already established because it is also one the assertions in the working memory.
- Now as all the parts of the antecedent in rule 4 are established so its consequent i.e., “raja is carnivorous” is established.
- The system now backtracks to rule 8 where in the second part of the antecedent says that X i.e., raja must have a big-mouth which now becomes the new hypothesis. This is already established because the system has an assertion that “raja has a big mouth”.
- Now as the whole antecedent of rule 8 is satisfied **so the system concludes that “raja is a lion”**.

We have seen that the system was able to work backward through the antecedent – consequent rules, using desired conclusions to decide that what assertions it should look for and ultimately establish the initial hypothesis.

How to choose the type of chaining among forward or backward chaining for a given problem?

Many rule-based deduction systems can chain either forward or backward, but determining which approach is better for a given problem is the point of discussion.

First, let us learn some basic things about rules i.e., **how a rule relates its input(s) (i.e., facts) to output(s) (i.e., conclusion)**. Whenever in a rule, a particular set of facts can lead to many conclusions, the rule is said to have a high degree of **fan out**, and is a strong candidate for backward chaining for its processing. On the other hand, whenever the rules are such that a particular hypothesis can lead to many questions for the hypothesis to be established, the rule is said to have a high degree of fan-in, and a high degree of **fan-in** is a strong candidate of forward chaining.

To summarize, the following points should help in choosing the type of chaining for reasoning purposes:

- If the set of facts that we already have or may establish can lead to a large number of conclusions or outputs, but the number of ways or input paths to reach that particular conclusion in which we are interested is small, then **the degree of fan out is greater than the degree of fan in. In such case, backward chaining is the preferred choice.**
- But, if the number of ways or input paths to reach the particular conclusion in which we are interested is large, but the number of conclusions that we can reach using the facts through that rule is small, then **the degree of fan in is greater than the degree of fan out. In such case, forward chaining is the preferred choice.**

For the case where **the degree of fan out and fan in are approximately the same**, then if not many facts are available and the problem is to check whether

one of the many possible conclusions is true, **backward chaining is the preferred choice**.

6.2.3 Conflict Resolution

Next, we discuss in detail some of the issues involved in a rule-based system.

Rule-based systems vary greatly in their details and syntax. A basic principle of a rule-based system is that each rule is an independent piece of knowledge. In an IF-THEN rule, the IF-part contains all the conditions for the application of the rule under consideration. The THEN-part tells the action to be taken by the interpreter. The interpreter need not search anywhere else except within the rule itself for the conditions required for application of the rule.

Another important consequence of the above-mentioned characteristic of a rule-based system is that no rule can call upon any other, and hence rules are ignorant and hence independent of each other. This gives a highly modular structure to the rule-based systems. Because of the highly modular structure of the rule-base, addition, deletion and modification of a rule can be done without any dangerous side effects.

The main problem with rule-based systems is that, when the rule base grows very large, checking whether a new rule intended to be added is redundant (i.e., already covered by some earlier rule) becomes difficult. Still worse, as the rule-base grows, checking the consistency of the rule-base also becomes quite difficult. By consistency, we mean there may be two rules having similar conditions, but the actions specified by the two rules conflict with each other.

Some of the conflict resolution strategies which are used to decide which rule to fire are given below:

- Don't fire a rule twice on the same data.
- Fire rules on more recent working memory elements before older ones. This allows the system to follow through a single chain of reasoning, rather than keep drawing new conclusions from old data.
- Fire rules with more specific preconditions before ones with more general preconditions. This allows us to deal with non-standard cases.

These strategies may help in achieving reasonable behavior from a forward-chaining system, but **the most important thing is how we should write the rules**. They should be carefully constructed, with the preconditions specifying as precisely as possible when different rules should fire. Otherwise, we will have little idea or control over what will happen.

To understand, let us take the following example in which we use forward chaining:

Example

Let us assume that the working memory initially contains the following facts :

(day monday)

(at-home ram)

(does-not-like ram)

Let, the existing set of rules are:

R1 : IF (day monday)
THEN ADD to working memory the fact : (working-with ram)

R2 : IF (day monday)
THEN ADD to working memory the fact : (talking-to ram)

R3 : IF (talking-to X) AND (working-with X)
THEN ADD to working memory the fact : (busy-at-work X)

R4 : IF (busy-at-work X) OR (at-office X)
THEN ADD to working memory the fact : (not-at-home X)

R5 : IF (not-at-home X)
THEN DELETE from working memory the fact : (happy X)

R6 : IF (working-with X)
THEN DELETE from working memory the fact : (does-not-like X)

Now, **to start the process of inference through forward chaining**, the rule-based system will first search for all the rule/s whose antecedent part/s are satisfied by the current set of facts in the working memory. *For example, in this example, we can see that the rules R1 and R2 are satisfied, so the system will chose one of them using its conflict resolution strategies.* Let the rule R1 be chosen. So (working-with ram) is added to the working memory (after substituting “ram” in place of X). So the working memory now looks like:

(working-with ram)

(day monday)

(at-home ram)

(does-not-like ram)

Now this cycle begins again. The system looks for rules that are satisfied and finds rules R2 and R6. Let the system choose rule R2. So now (talking-to ram) is added to the working memory. So now the working memory contains the following:

(talking-to ram)

(working-with ram)

(day monday)

(at-home ram)

(does-not-like ram)

Now in the next cycle, rule R3 fires, so now (busy-at-work ram) is added to working memory, which now looks like:

(busy-at-work ram)
(talking-to ram)
(working-with ram)
(day monday)
(at-home ram)
(does-not-like ram)

Now antecedent parts of rules R4 and R6 are satisfied. Let rule R4 fires, so (not-at-home, ram) is added to working memory which now looks like :

(not-at-home ram)
(busy-at-work ram)
(talking-to ram)
(working-with ram)
(day monday)
(at-home ram)
(does-not-like ram)

In the next cycle, rule R5 fires so (at-home ram) is removed from the working memory :

(not-at-home ram)
(busy-at-work ram)
(talking-to ram)
(working-with ram)
(day monday)
(does-not-like ram)

The forward chaining will continue like this. But we have to be sure of one thing – the ordering of the rules firing is important. A change in the order of rule firing may result in a different working memory.

Check your Progress - 1

1. In the “Animal Identifier System” discussed above, use forward chaining to try to identify the animal called “raja”.

.....
.....
.....

6.3 SEMANTIC NETS

Semantic Network representations provide a **structured knowledge representation**. In such a network, parts of knowledge are clustered into semantic groups. In semantic

networks, the concepts and entities/objects of the problem domain are represented by nodes and relationships between these entities are generally shown by directed arrows. A semantic network representation is a pictorial depiction of objects, their attributes, and the relationships that exist between these objects and other entities. A semantic net is just a graph, where the nodes in the graph represent concepts, and the arcs are labeled and represent binary relationships between concepts. These networks provide a more natural way of mapping to and from natural language than other representation schemes.

For example, the fact (a piece of knowledge): **Mohan struck Nita in the garden with a sharp knife last week**, is represented by the semantic network shown in Figure 6.1.

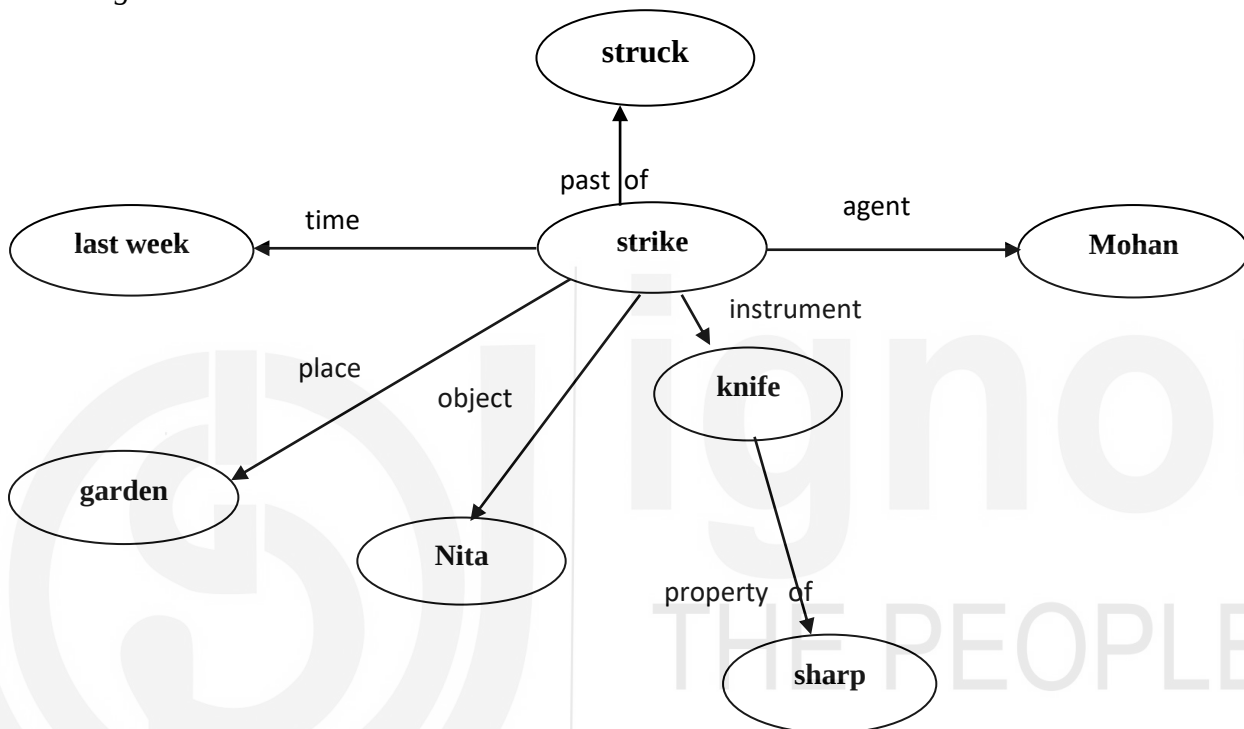


Figure 6.1 Semantic Network

The two most important relations between concepts are (i) *subclass* relation between a class and its superclass, and (ii) *instance* relation between an object and its class. Other relations may be *has-part*, *color* etc. As mentioned earlier, relations are indicated by labeled arcs.

As information in semantic networks is clustered together through relational links, the knowledge required for the performance of some task is generally available within a short spatial span of the semantic network. This type of knowledge organisation in some way, resembles the way knowledge is stored and retrieved by human beings.

Subclass and *instance* relations allow us to use **inheritance** to infer new facts/relations from the explicitly represented ones. We have already mentioned that **the graphical portrayal of knowledge in semantic networks**, being visual, is easier than other representation schemes for human beings to comprehend. This fact helps human beings guide the expert system whenever required. This is perhaps the reason for the popularity of semantic networks.

Check Your Progress – 2

2. Draw a semantic network for the following English statement:

Mohan struck Nita and Nita's mother struck Mohan.

.....

.....

.....

6.4 FRAMES

Frames are a variant of semantic networks that are one of the popular ways of representing non-procedural knowledge in an expert system. In a frame, all the information relevant to a particular concept is stored in a single complex entity, called a frame. Frames look like the data structure, called arecord. Frames support inheritance. They are often used to capture knowledge about *typical* objects or events, such as a car, or even a mathematical object like rectangle. As mentioned earlier, a frame is a structured object and different names like *Schema*, *Script*, *Prototype*, and even *Object* are used in stead of frame, in computer science literature.

We may represent some knowledge about a lion in frames as follows:

Mammal :

Subclass : Animal

warm_blooded : yes

Lion :

subclass : Mammal

eating-habbit : carnivorous

size : medium

Raja :

instance : Lion

colour : dull-Yellow

owner : Amar Circus

Sheru :

instance : Lion

size : small

A particular frame (such as Lion) has a number of *attributes* or *slots*, such as *eating-habit* and *size*. Each of these slots may be filled with particular values, such as the *eating-habit* for lion may be filled up as *carnivorous*.

Sometimes a slot contains additional information, such as how to apply or use the slot values. Typically, a slot contains information such as (*attribute, value*) pairs, default values, conditions for filling a **slot**, pointers to other related frames, and also procedures that are activated when needed for different purposes.

In the case of frame representation of knowledge, **inheritance is simple** if an object has a single parent class and if each slot takes a single value. For example, if a mammal is warm blooded, then automatically, a lion being a mammal will also be warm blooded.

But **in the case of multiple inheritance**, i.e., in the case of an object having more than one parent class, we have to decide which parent to inherit from. For example, a lion may inherit from “wild animals” or “circus animals”. In general, both the slots and slot values may themselves be frames and so on.

Frame systems are pretty complex and sophisticated knowledge representation tools. This representation has become so popular that special high level frame based representation languages have been developed. Most of these languages use LISP as the host language. It is also possible to represent frame-like structures using object oriented programming languages, extensions to the programming language LISP.

Check Your Progress – 3

3. Define a frame for the entity *date* which consists of *day*, *month* and *year*. each of which is a number with restrictions which are well-known. Also a procedure named *compute-day-of-week* is already defined.

.....

.....

.....

6.5 SCRIPTS

A script is a structured representation describing a stereotyped sequence of events in a particular context.

Scripts are used in natural language understanding systems to organize a knowledge base in terms of the situations that the system should understand. Scripts use a frame-like structure to represent the commonly occurring experience like going to the movies eating in a restaurant, shopping in a supermarket, or visiting an ophthalmologist.

Thus, a script is a structure that prescribes a set of circumstances that could be expected to follow on from one another.

Scripts are beneficial because:

- Events tend to occur in known runs or patterns.
- A causal relationship between events exist.
- An entry condition exists which allows an event to take place.
- Prerequisites exist for events to take place.

Components of a script

The components of a script include:

- **Entry condition:** These are basic conditions that must be fulfilled before events in the script can occur.
- **Results:** Conditions that will be true after the events in the script have occurred.
- **Props:** Slots representing objects involved in events
- **Roles:** These are the actions that the individual participants perform.
- **Track:** Variations on the script. Different tracks may share components of the same scripts.
- **Scenes:** The sequence of events that occur.

When describing a script, special action symbols are used. These are:

Symbol	Meaning	Example
ATRANS	transfer a relationship	give
PTRANS	transfer physical location of an object	go
PROPEL	apply physical force to an object	push
MOVE	move body part by owner	kick
GRASP	grab an object by an actor	hold
INGEST	taking an object by an animal eat	drink
EXPEL	expel from animal's body	cry
MTRANS	transfer mental information	tell
MBUILD	mentally make new information	decide
CONC	conceptualize or think about an idea	think
SPEAK	produce sound	Say
ATTEND	focus sense organ	listen

Example:-Script for going to the bank to withdraw money.

SCRIPT : Withdraw money

TRACK : Bank

PROPS : Money

Counter

Form

Token

Roles :

P= Customer

E= Employee

C= Cashier

Entry conditions: P has no or less money.

The bank is open.

Results : P has more money.

Scene 1: Entering

P PTRANS P into the Bank

P ATTEND eyes to E

P MOVE P to E

Scene 2: Filling form

P MTRANS signal to E

E ATRANS form to P

P PROPEL form for writing

P ATRANS form to P

ATrans form to P

Scene 3: Withdrawing money

P ATTEND eyes to counter

P PTRANS P to queue at the counter

P PTRANS token to C

C ATRANS money to P

Scene 4: Exiting the bank

P PTRANS P to out of bank

Advantages of Scripts

- Ability to predict events.
- A single coherent interpretation maybe builds up from a collection of observations.

Disadvantages of Scripts

- Less general than frames.
- May not be suitable to represent all kinds of knowledge

6.6 SUMMARY

This unit mainly discussed the various knowledge representation mechanisms, used in Artificial Intelligence. The unit began with the discussion on Rule-Based Systems, and discussed the related concept of Forward Chaining and Backward Chaining, later the concept of Conflict Resolution was discussed. The unit also discussed the other techniques of knowledge representation like Semantic nets, Frames and Scripts; along with relevant examples for each.

6.7 SOLUTIONS/ANSWERS

Check Your Progress – 1

Exercise 1: Refer to section 6.2

Check Your Progress – 2

Exercise 2: Refer to section 6.3

Check Your Progress – 3

Exercise 3: Refer to section 6.4

6.8 FURTHER READINGS

1. Ela Kumar, “Artificial Intelligence”, IK International Publications
2. E. Rich and K. Knight, “Artificial intelligence”, Tata Mc Graw Hill Publications
3. N.J. Nilsson, “Principles of AI”, Narosa Publ. House Publications
4. John J. Craig, “Introduction to Robotics”, Addison Wesley publication
5. D.W. Patterson, “Introduction to AI and Expert Systems” Pearson publication

